
神经语言模型的 scaling laws

Scaling Laws for Neural Language Models

Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess,
Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, Dario Amodei

Johns Hopkins University · OpenAI

arXiv:2001.08361v1 · 2020 年 1 月 23 日

摘要

我们研究语言模型在交叉熵损失上的性能所遵循的经验 scaling laws。损失随模型规模、数据集大小和训练所用计算量呈幂律缩放，其中一些趋势横跨七个以上的数量级。在很宽的范围内，网络宽度或深度等其他架构细节的影响很小。简单的方程刻画了过拟合对模型/数据集大小的依赖关系，以及训练速度对模型规模的依赖关系。这些关系使我们能够确定固定计算预算的最优分配方式。更大模型的样本效率显著更高，因此，计算效率最优的训练方式是在相对适量的数据上训练非常大的模型，并在远未收敛时就停止训练。

贡献说明

贡献：Jared Kaplan 和 Sam McCandlish 主导了这项研究。Tom Henighan 完成了 LSTM 实验。Tom Brown、Rewon Child、Scott Gray 和 Alec Radford 开发了经过优化的 Transformer 实现。Jeff Wu、Benjamin Chess 和 Alec Radford 开发了文本数据集。Dario Amodei 在整个项目期间提供了指导。

目录

1 引言	1
1.1 总结	1
1.2 Scaling Laws 总结	2
1.3 符号约定	3
2 背景和方法	4
2.1 Transformer 参数量与计算量的缩放关系	4
2.2 训练流程	4
2.3 数据集	5
3 经验结果和基本幂律	5
3.1 Transformer 形状和超参数的近似独立性	5
3.2 性能与 non-embedding parameter count N 的关系	5
3.3 性能与数据集大小及计算量的关系	6
4 描绘无限数据极限与过拟合	7
4.1 提议的 $L(N, D)$ 公式	7
4.2 结果	8
5 模型规模与训练时间的 Scaling Laws	8
5.1 针对在 $B_{\text{crit}}(L)$ 下训练的校正	9
5.2 $L(N, S_{\min})$ 的结果以及性能如何随模型规模和计算量变化	10
5.3 Early Stopping 步数的下界	10
6 计算预算的最优分配	11
6.1 最优性能与分配	11
6.2 来自 $L(N, S_{\min})$ 的预测	12
6.3 矛盾与一个猜想	12
7 相关工作	13
8 讨论	14
附录	15
A 幂律总结	15
B 计算高效前沿的经验模型	15
B.1 定义方程	15
B.2 高效训练	16
B.3 与低效训练的比较	16
B.4 次优模型规模	17
C 注意事项	17
D 补充图	17
D.1 Early Stopping 以及测试与训练的比较	17
D.2 通用 Transformer	17
D.3 Batch Size	18
D.4 样本效率与模型规模	18
D.5 上下文依赖性	19
D.6 learning-rate schedule 与误差分析	20
D.7 拟合细节与幂律质量	21
D.8 泛化与架构	21

1 引言

语言为人工智能研究提供了一个自然的领域，因为绝大多数推理任务都能以语言高效地表达和评估，而世界上的文本也为通过生成式建模进行无监督学习提供了丰富的数据。最近，深度学习在语言建模方面取得了迅速进展，最先进的模型 [RNSS18, DCLT18, YDY⁺19, LOG⁺19, RSR⁺19] 在许多具体任务上正接近人类水平的性能 [WPN⁺19]，其中包括生成连贯的多段落提示文本样本 [RWC⁺19]。

人们或许会预期，语言建模性能取决于模型架构、神经模型的规模、用于训练模型的计算能力，以及可供这一训练过程使用的数据。在本研究中，我们将从经验上探究语言建模损失对所有这些因素的依赖关系，并重点关注 Transformer 架构 [VSP⁺17, LSP⁺18]。语言任务性能的上限很高、下限很低，这使我们能够研究尺度横跨七个以上数量级的趋势。

在全文中，我们将观察到性能随训练时间、上下文长度、数据集大小、模型规模和计算预算变化的精确幂律缩放关系。

1.1 总结

我们对 Transformer 语言模型的主要发现如下：

性能强烈依赖尺度，而很少依赖模型形状：模型性能最强烈地依赖于尺度，尺度由三个因素构成：模型参数数量 N （不包括 embeddings）、数据集大小 D ，以及训练所用的计算量 C 。在合理范围内，性能对深度与宽度等其他架构超参数的依赖非常弱。（第 3 节）

平滑的幂律：当性能不受另外两个因素制约时，它与三个尺度因素 N, D, C 中的每一个都呈幂律关系，相关趋势横跨六个以上的数量级（见图 1）。在高端，我们没有观察到偏离这些趋势的迹象，不过性能最终必然会在达到零损失之前趋于平坦。（第 3 节）

过拟合的普适性：只要同步扩大 N 和 D ，性能就会以可预测的方式提升；但如果固定 N 或 D 中的一个而增大另一个，就会进入收益递减的区域。性能损失以可预测的方式取决于比率 $N^{0.74}/D$ ，这意味着模型规模每扩大 8 倍，只需将数据增加约 5 倍即可避免性能损失。（第 4 节）

训练的普适性：训练曲线遵循可预测的幂律，其参数大致不依赖于模型规模。通过外推训练曲线的早期部分，我们可以粗略预测，如果继续训练很长时间，模型将达到怎样的损失。（第 5 节）

迁移能力随测试性能提升：当我们在与训练文本分布不同的文本上评估模型时，结果与训练分布验证集上的结果高度相关，二者的损失之间存在大致恒定的偏移——换言之，迁移到不同分布会带来恒定的性能损失，但除此之外，其改善程度大体与训练集上的性能同步。（第 3.2.2 节）

样本效率：大模型的样本效率比小模型更高，使用更少的优化步骤（图 2）和更少的数据点（图 4）即可达到相同的性能水平。

训练至收敛是低效的：当我们在固定计算预算 C 内工作，但模型规模 N 或可用数据 D 不受任何其他限制时，可以通过训练非常大的模型并在远未收敛时停止来获得最优性能（见图 3）。因此，相较于根据小模型训练至收敛所得出的预期，计算效率最高的训练会具有高得多的样本效率；其数据需求随训练计算量仅以 $D \sim C^{0.27}$ 的速度非常缓慢地增长。（第 6 节）

最优 batch size：训练这些模型的理想 batch size 大致仅为损失的幂函数，并且仍然可以通过测量 gradient noise scale [MKAT18] 来确定；对于我们能够训练的最大模型，收敛时的理想 batch size 约为 100 万至 200 万个 tokens。（第 5.1 节）

¹这里展示的是使用足够小的 batch size 时预测的计算量。与纯经验数据的比较见图 13。

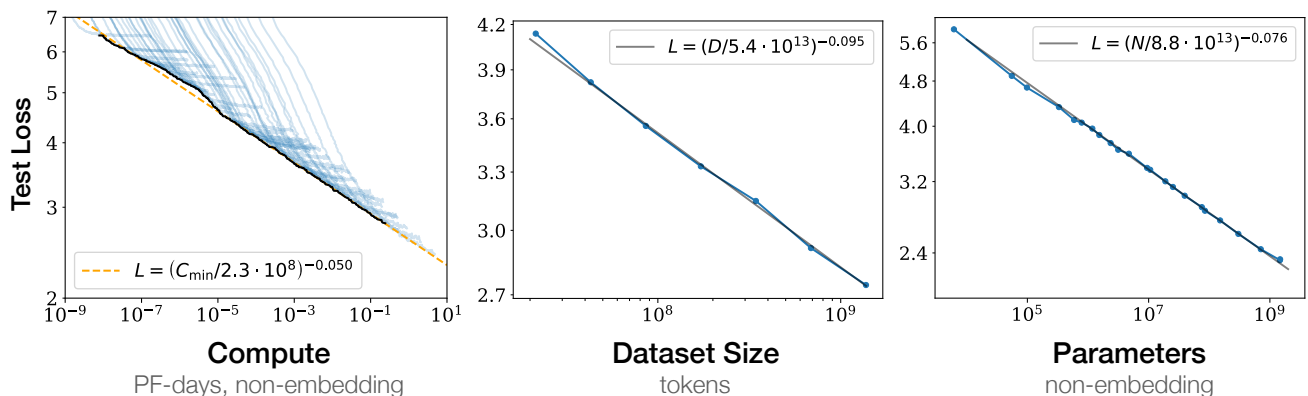


图 1 随着模型规模、数据集大小和训练所用计算量¹的增加，语言建模性能会平滑提升。为获得最优性能，必须同步扩大这三个因素。当性能不受另外两个因素制约时，经验性能与每个单独因素之间都呈幂律关系。

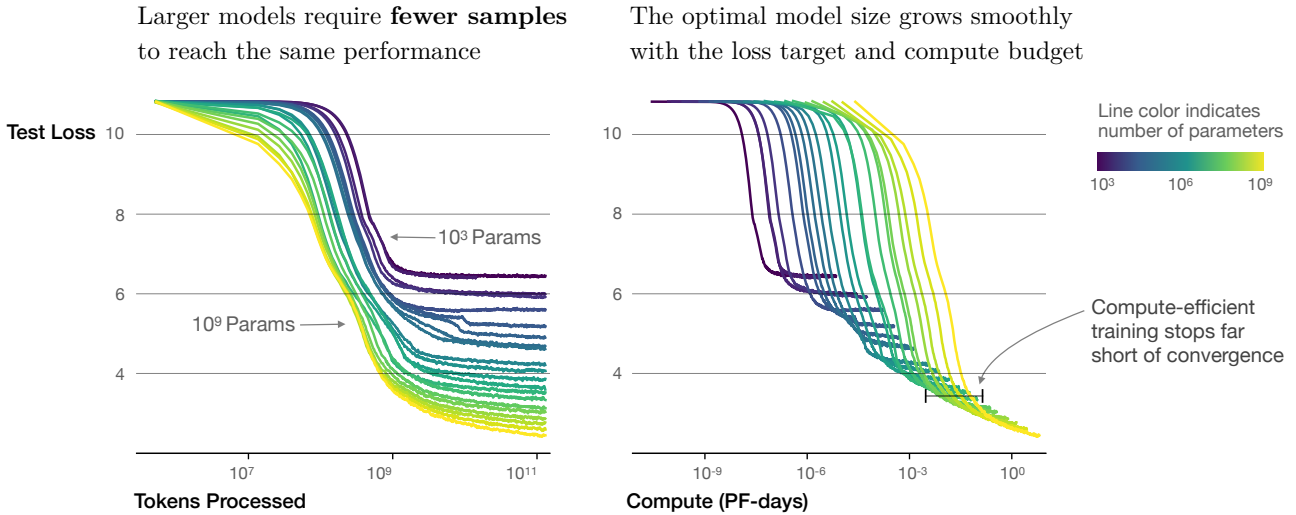


图 2 我们展示了一系列语言模型训练过程，模型规模从 10^3 到 10^9 个参数（不包括 embeddings）不等。

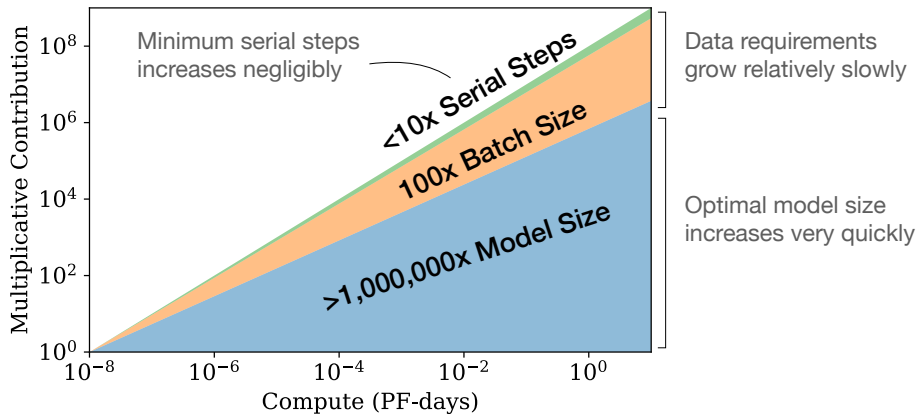


图 3 随着可用计算量增加，我们可以选择分配多少计算量来训练更大的模型、使用更大的批量，以及训练更多的步骤。这里以计算量增加十亿倍为例进行说明。对于计算效率最优的训练，大部分增量都应分配给模型规模的扩大。只需相对少量地增加数据即可避免数据复用。在增加的数据中，大部分可以通过更大的 batch size 用于提高并行度，而所需的串行训练时间只需增加非常少。

综合来看，这些结果表明，只要适当地扩大模型规模、数据和计算量，语言建模性能就会平滑且可预测地提升。我们预计，更大的语言模型将比当前模型表现得更好，并且样本效率更高。

1.2 Scaling Laws 总结

对于经过训练、以 autoregressive 方式对语言建模的 Transformer，当性能只受 non-embedding parameters 的数量 N 、数据集大小 D 或经过最优分配的计算预算 $C_{\min}$ 三者之一限制时，其测试损失可以用幂律预测（见图 1）：

1. 对于参数数量有限、在足够大的数据集上训练至收敛的模型：

$$L(N) = (N_c/N)^{\alpha_N}; \quad \alpha_N \sim 0.076, \quad N_c \sim 8.8 \times 10^{13} \text{ (non-embedding parameters)} \quad (1.1)$$

2. 对于使用有限数据集并通过 early stopping 进行训练的大模型：

$$L(D) = (D_c/D)^{\alpha_D}; \quad \alpha_D \sim 0.095, \quad D_c \sim 5.4 \times 10^{13} \text{ (tokens)} \quad (1.2)$$

3. 当使用有限的计算量、足够大的数据集、规模最优的模型以及足够小的 batch size 进行训练时（从而最优地利用²计算量）：

$$L(C_{\min}) = (C_c^{\min}/C_{\min})^{\alpha_C^{\min}}; \quad \alpha_C^{\min} \sim 0.050, \quad C_c^{\min} \sim 3.1 \times 10^8 \text{ (PF-日)} \quad (1.3)$$

这些关系在 $C_{\min}$ 横跨八个数量级、 N 横跨六个数量级、 D 横跨两个以上数量级的范围内均成立。它们对模型形状和其他 Transformer 超参数（深度、宽度、自注意力头数量）的依赖非常弱，其中的具体数值与 Webtext2

²在固定 batch size 下进行训练时，我们也观察到训练计算量 C （图 1）所遵循的经验幂律趋势，但进行预测时应使用 $C_{\min}$ 所遵循的趋势。二者通过方程 (5.5) 联系起来。

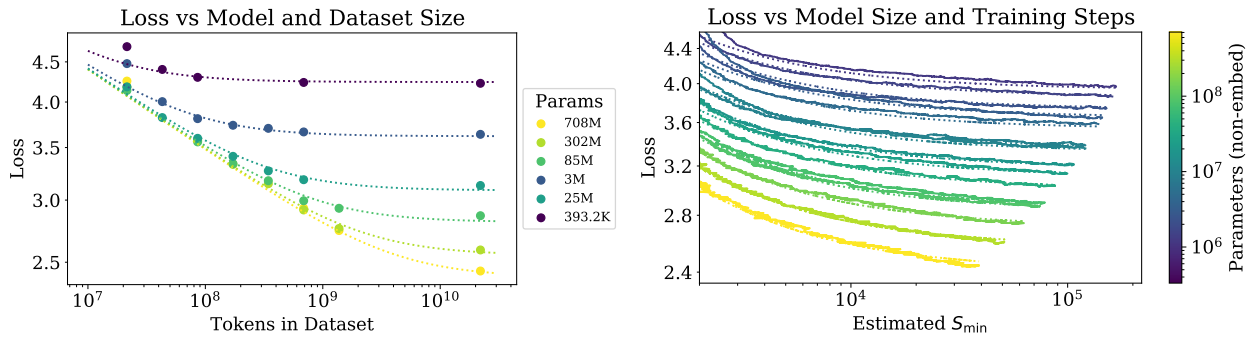


图 4 左图：根据方程 (1.5)，early-stopped 测试损失 $L(N, D)$ 随数据集大小 D 和模型规模 N 以可预测的方式变化。右图：在最初的瞬态阶段之后，所有模型规模 N 的学习曲线都可以用方程 (1.6) 拟合；该方程以 $S_{\min}$ 为参数，后者是在大 batch size 下训练时的步骤数（详见第 5.1 节）。

训练集 [RWC⁺19] 有关。幂律 $\alpha_N, \alpha_D, \alpha_C^{\min}$ 指定了在扩大 N 、 D 或 $C_{\min}$ 时预期的性能提升程度；例如，将参数数量加倍，会使损失缩小至原来的 $2^{-\alpha_N} = 0.95$ 倍。 $N_c, C_c^{\min}$ 和 D_c 的精确数值取决于词表大小和 tokenization 方式，因此不具有根本意义。

决定数据并行中速度/效率权衡的 critical batch size ([MKAT18]) 也大致服从关于 L 的幂律：

$$B_{\text{crit}}(L) = \frac{B_*}{L^{1/\alpha_B}}, \quad B_* \sim 2 \cdot 10^8 \text{ 个 tokens}, \quad \alpha_B \sim 0.21 \quad (1.4)$$

方程 (1.1) 和 (1.2) 共同表明，随着模型规模增加，我们应按照 $D \propto N^{\frac{\alpha_N}{\alpha_D}} \sim N^{0.74}$ 以次线性方式增大数据集。事实上，我们发现，存在一个将 (1.1) 和 (1.2) 结合起来的单一方程，它刻画了对 N 和 D 的同时依赖关系，并支配过拟合的程度：

$$L(N, D) = \left[\left(\frac{N_c}{N} \right)^{\frac{\alpha_N}{\alpha_D}} + \frac{D_c}{D} \right]^{\alpha_D} \quad (1.5)$$

其拟合结果如图 4 左侧所示。我们推测，这一函数形式也可用于参数化其他生成式建模任务训练后的对数似然。

在无限数据极限下，用有限的参数更新步骤数 S 训练给定模型时，经过最初的瞬态阶段之后，可以用下式精确拟合学习曲线（见图 4 右侧）

$$L(N, S) = \left(\frac{N_c}{N} \right)^{\alpha_N} + \left(\frac{S_c}{S_{\min}(S)} \right)^{\alpha_S} \quad (1.6)$$

其中 $S_c \approx 2.1 \times 10^3$ ， $\alpha_S \approx 0.76$ ，而 $S_{\min}(S)$ 是可能的最少优化步骤数（参数更新次数），其值使用方程 (5.4) 进行估计。

当在固定计算预算 C 内进行训练，但没有其他约束时，方程 (1.6) 可以预测，最优模型规模 N 、最优 batch size B 、最优步骤数 S 和数据集大小 D 应按如下方式增长：

$$N \propto C^{\alpha_C^{\min}/\alpha_N}, \quad B \propto C^{\alpha_C^{\min}/\alpha_B}, \quad S \propto C^{\alpha_C^{\min}/\alpha_S}, \quad D = B \cdot S \quad (1.7)$$

其中

$$\alpha_C^{\min} = 1 / (1/\alpha_S + 1/\alpha_B + 1/\alpha_N) \quad (1.8)$$

这与经验最优结果 $N \propto C_{\min}^{0.73}$ 、 $B \propto C_{\min}^{0.24}$ 和 $S \propto C_{\min}^{0.03}$ 非常吻合。随着计算预算 C 增加，它应主要用于更大的模型，而无需大幅增加训练时间或数据集大小（见图 3）。这也意味着随着模型变大，它们的样本效率会越来越高。在实践中，由于硬件约束，研究人员通常会采用比计算效率最高的配置更小的模型，并训练更长时间。最优性能以幂律方式取决于总计算量（见方程 (1.3)）。

我们为方程 (1.5) 提供了一些基本的理论依据，分析了学习曲线拟合及其对训练时间的影响，并按 token 细分了我们的结果。我们还将这些结果与 LSTM 和循环 Transformer [DGV⁺18] 进行了简要比较。

1.3 符号约定

我们使用以下符号：

- L ——以自然信息单位（纳特）计的交叉熵损失。通常会对一个上下文中的 tokens 取平均值，但在某些情况下，我们会报告上下文中特定 tokens 的损失。
- N ——模型参数数量，不包括所有 *vocabulary embeddings* 和 *positional embeddings*
- $C \approx 6NBS$ ——对 non-embedding 训练总计算量的估计，其中 B 是 batch size， S 是训练步骤数（即参数更新次数）。我们以 PF-日为单位给出数值，其中一个 PF-日 = $10^{15} \times 24 \times 3600 = 8.64 \times 10^{19}$ 次浮点运算。
- D ——以 tokens 计的数据集大小

操作	参数	每个 token 的 FLOPs
Embedding	$(n_{\text{vocab}} + n_{\text{ctx}}) d_{\text{model}}$	$4d_{\text{model}}$
注意力: QKV	$n_{\text{layer}} d_{\text{model}} 3d_{\text{attn}}$	$2n_{\text{layer}} d_{\text{model}} 3d_{\text{attn}}$
注意力: 掩码	—	$2n_{\text{layer}} n_{\text{ctx}} d_{\text{attn}}$
注意力: 投影	$n_{\text{layer}} d_{\text{attn}} d_{\text{model}}$	$2n_{\text{layer}} d_{\text{attn}} d_{\text{embd}}$
前馈	$n_{\text{layer}} 2d_{\text{model}} d_{\text{ff}}$	$2n_{\text{layer}} 2d_{\text{model}} d_{\text{ff}}$
De-embed	—	$2d_{\text{model}} n_{\text{vocab}}$
总计 (Non-Embedding)	$N = 2d_{\text{model}} n_{\text{layer}} (2d_{\text{attn}} + d_{\text{ff}})$	$C_{\text{forward}} = 2N + 2n_{\text{layer}} n_{\text{ctx}} d_{\text{attn}}$

表 1 Transformer 模型的参数量和计算量（前向传播）估计。省略了非线性、偏置和层归一化等次要项。

- B_{crit} ——critical batch size [MKAT18]，其定义和讨论见第 5.1 节。使用 critical batch size 进行训练，可以在时间效率与计算效率之间实现大致最优的折中。
- C_{min} ——为达到给定损失值所需的最少 non-embedding 计算量的估计。如果模型采用远小于 critical batch size 的 batch size 进行训练，这就是会使用的训练计算量。
- S_{min} ——为达到给定损失值所需的最少训练步骤数的估计。这也是在模型采用远大于 critical batch size 的 batch size 进行训练时会使用的训练步骤数。
- α_X ——损失按 $L(X) \propto 1/X^{\alpha_X}$ 缩放时的幂律指数，其中 X 可以是 $N, D, C, S, B, C^{\text{min}}$ 中的任意一个。

2 背景和方法

我们在 WebText [RWC⁺19] 数据集的扩展版本 WebText2 上训练语言模型；该数据集使用字节对编码 [SHB15] 进行 tokenization，词表大小为 $n_{\text{vocab}} = 50257$ 。我们优化在包含 1024 个 tokens 的上下文上取平均值的 autoregressive 对数似然（即交叉熵损失），这也是我们的主要性能指标。我们记录模型在 WebText2 测试分布以及若干其他文本分布上的损失。我们主要训练仅含解码器的 [LSP⁺18, RNSS18] Transformer [VSP⁺17] 模型，不过也训练了 LSTM 模型和通用 Transformer [DGV⁺18] 用于比较。

2.1 Transformer 参数量与计算量的缩放关系

我们使用超参数 n_{layer} （层数）、 d_{model} （残差流的维度）、 d_{ff} （中间前馈层的维度）、 d_{attn} （注意力输出的维度）和 n_{heads} （每层的注意力头数）来参数化 Transformer 架构。我们在输入上下文中包含 n_{ctx} 个 tokens；除非另有说明，否则 $n_{\text{ctx}} = 1024$ 。

我们用 N 表示模型规模，并将其定义为 *non-embedding parameters* 的数量

$$\begin{aligned} N &\approx 2d_{\text{model}} n_{\text{layer}} (2d_{\text{attn}} + d_{\text{ff}}) \\ &= 12n_{\text{layer}} d_{\text{model}}^2, \text{ 在标准设定下 } d_{\text{attn}} = d_{\text{ff}}/4 = d_{\text{model}} \end{aligned} \quad (2.1)$$

其中我们排除了偏置和其他次要项。我们的模型在 embedding 矩阵中还包含 $n_{\text{vocab}} d_{\text{model}}$ 个参数，并使用 $n_{\text{ctx}} d_{\text{model}}$ 个参数表示位置 embeddings，但在讨论“模型规模” N 时，我们不包括这些参数；我们将看到，这样做会得到明显更简洁的 scaling laws。

对 Transformer 执行一次前向传播大致涉及

$$C_{\text{forward}} \approx 2N + 2n_{\text{layer}} n_{\text{ctx}} d_{\text{model}} \quad (2.2)$$

次加乘运算，其中系数 2 来自矩阵乘法所使用的乘加运算。表 1 中给出了按操作划分的更详细参数量和计算量。

对于满足 $d_{\text{model}} > n_{\text{ctx}}/12$ 的上下文和模型，每个 token 中依赖上下文的计算成本只占总计算量中相对较小的一部分。由于我们主要研究 $d_{\text{model}} \gg n_{\text{ctx}}/12$ 的模型，因此在估算训练计算量时不包括依赖上下文的项。计入反向传播（其计算量约为前向传播的两倍）后，我们将估计的 non-embedding 计算量定义为每个训练 token $C \approx 6N$ 次浮点运算。

2.2 训练流程

除非另有说明，否则我们使用 Adam 优化器 [KB14] 对模型进行固定 2.5×10^5 步的训练，batch size 为 512 个序列，每个序列包含 1024 个 tokens。由于内存限制，我们最大的模型（超过 10 亿个参数）使用 Adafactor [SS18] 进行训练。我们试验了多种 learning rate 和调度方案，相关讨论见附录 D.6。我们发现，收敛时的结果基本不依赖于 learning-rate schedule。除非另有说明，否则数据中包含的所有训练过程都采用一种 learning-rate schedule 方案：先进行 3000 步线性预热，随后按余弦方式衰减至零。

2.3 数据集

我们在 [RWC⁺19] 所述 WebText 数据集的扩展版本上训练模型。原始 WebText 数据集通过抓取截至 2017 年 12 月、获得至少 3 点社区积分的 Reddit 外链而得到。在第二个版本 WebText2 中，我们加入了 2018 年 1 月至 10 月期间的 Reddit 外链，同样要求社区积分至少为 3 点。社区积分阈值被用作判断人们是否认为链接有趣或有用的启发式指标。新链接中的文本是使用 Newspaper3k Python 库提取的。整个数据集共包含 2030 万篇文档、96 GB 文本和 1.62×10^{10} 个单词（按 wc 的定义）。随后，我们应用 [RWC⁺19] 中所述的可逆 tokenizer，得到 2.29×10^{10} 个 tokens。我们留出其中的 6.6×10^8 个 tokens 用作测试集，并且还在以类似方式制备的图书语料库 [ZKZ⁺15]、公共网页抓取语料库 [Fou]、英文维基百科和公开可用的互联网图书集合样本上进行测试。

3 经验结果和基本幂律

为了刻画语言模型的缩放特性，我们训练了多种多样的模型，并改变了若干因素，其中包括：

- 模型规模（从 768 个到 15 亿个 non-embedding parameters）
- 数据集大小（从 2200 万到 230 亿个 tokens）
- 形状（包括深度、宽度、注意力头和前馈维度）
- 上下文长度（大多数训练过程为 1024，不过我们也试验了更短的上下文）
- batch size（大多数训练过程为 2^{19} ，但我们也通过改变它来测量 critical batch size）

本节将展示数据和受经验启发的拟合，并将理论分析留至后面的章节。

3.1 Transformer 形状和超参数的近似独立性

在我们保持 non-embedding parameter count N 不变时，Transformer 性能对形状参数 n_{layer} , n_{heads} 和 d_{ff} 的依赖非常弱。为了得到这些结果，我们在固定模型规模的同时改变单个超参数来训练模型。对于 n_{heads} ，这一操作最为简单。在改变 n_{layer} 时，我们同时改变 d_{model} ，同时保持 $N \approx 12n_{\text{layer}}d_{\text{model}}^2$ 不变。同样，为了在固定模型规模的情况下改变 d_{ff} ，我们也按表 1 中参数量所要求的方式同时改变 d_{model} 参数。如果更深的 Transformer 实际上表现得像较浅模型的集成，就会得到 n_{layers} 的独立性，正如针对 ResNet 所提出的观点 [VWB16]。结果如图 5 所示。

3.2 性能与 non-embedding parameter count N 的关系

在图 6 中，我们展示了众多不同模型的性能，范围从形状为 $(n_{\text{layer}}, d_{\text{model}}) = (2, 128)$ 的小模型，一直到形状介于 $(6, 4288)$ 和 $(207, 768)$ 之间的十亿参数模型。这里，我们在完整的 WebText2 数据集上将模型训练到接近收敛，并且没有观察到过拟合（最大的模型可能除外）。

如图 1 所示，我们发现 non-embedding parameter count N 呈现稳定趋势，可以用方程 (1.5) 的第一项拟合，即

$$L(N) \approx \left(\frac{N_c}{N} \right)^{\alpha_N} \quad (3.1)$$

要观察到这些趋势，将性能视为 N 的函数进行研究至关重要；如果改用总参数数量（包括 embedding parameters），这一趋势就会在一定程度上变得模糊（见图 6）。这表明可以在不影响性能的情况下缩小 embedding 矩阵，近期研究 [LCG⁺19] 中也观察到了这一点。

尽管这些模型是在 WebText2 数据集上训练的，但如图 8 所示，它们在其他多种数据集上的测试损失也与 N 呈幂律关系，并且幂指数几乎相同。

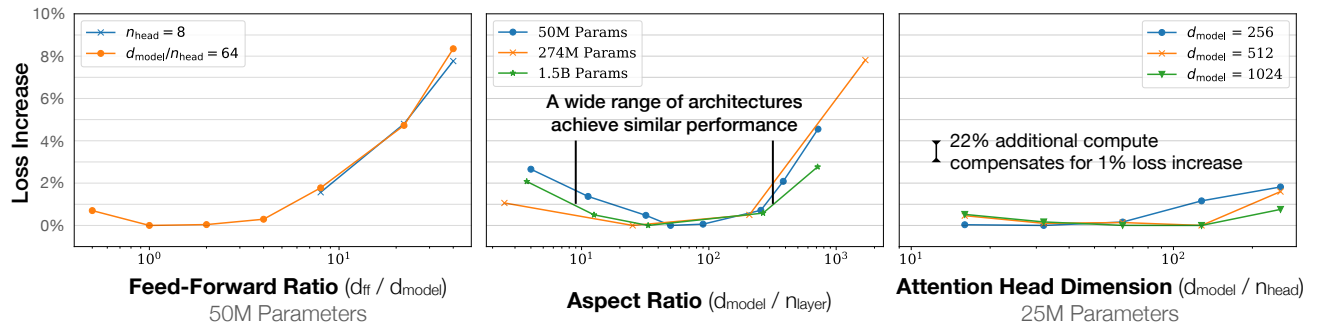


图 5 当 non-embedding parameters 总数 N 保持不变时，性能对模型形状的依赖非常弱。在很大的形状范围内，损失的变化只有百分之几。参数数量的微小差异通过使用 $L(N)$ 的拟合作为基线来补偿。尤其是，纵横比即使相差 40 倍，对性能的影响也很小； $(n_{\text{layer}}, d_{\text{model}}) = (6, 4288)$ 所达到的损失与 [RWC⁺19] 中使用的 $(48, 1600)$ 模型相差不到 3%。

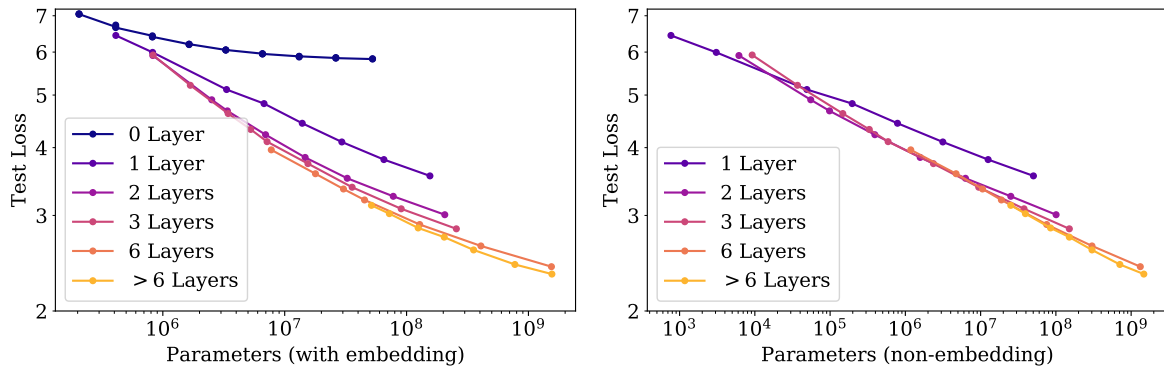


图 6 左图：当我们包括 embedding parameters 时，除了参数数量之外，性能似乎还强烈依赖于层数。右图：当我们排除 embedding parameters 时，不同深度模型的性能会收敛到同一趋势。只有层数少于 2 层或深宽比极端的模型才会显著偏离这一趋势。

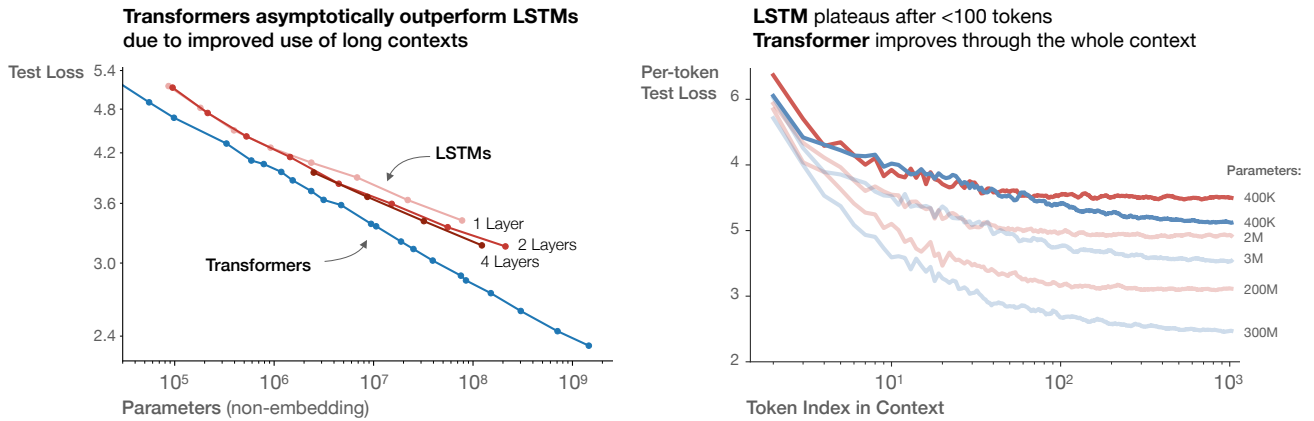


图 7

3.2.1 与 LSTM 和通用 Transformer 的比较

在图 7 中，我们比较了 LSTM 和 Transformer 的性能随 non-embedding parameter count N 的变化。LSTM 使用相同的数据集和上下文长度进行训练。从这些图中可以看到，对于出现在上下文前部的 tokens，LSTM 的表现与 Transformer 一样好，但对于后部的 tokens，它们无法达到 Transformer 的性能。在附录 D.5 中，我们给出了性能与上下文位置之间的幂律关系；对于更大的模型，幂指数越来越大，这表明其快速识别模式的能力得到提升。

我们还比较了标准 Transformer 与循环 Transformer [DGV⁺18] 的性能，结果见附录中的图 17。这些模型会复用参数，因此作为 N 的函数，其性能略好，但代价是每个参数需要额外的计算量。

3.2.2 数据分布之间的泛化

我们还在一组额外的文本数据分布上测试了模型。这些数据集上的测试损失随模型规模的变化如图 8 所示；在所有情况下，模型都只在 WebText2 数据集上进行了训练。可以看到，其他数据分布上的损失随模型规模平滑改善，与 WebText2 上的改善完全同步。我们发现，泛化几乎只取决于分布内验证损失，而不取决于训练持续时间或接近收敛的程度。我们也没有观察到它对模型深度的依赖（见附录 D.8）。

3.3 性能与数据集大小及计算量的关系

我们在图 1 中展示了测试损失随数据集大小 D （以 tokens 计）和训练计算量 C 变化的经验趋势。

为了研究随 D 变化的趋势，我们在 WebText2 数据集的固定子集上训练了一个 $(n_{\text{layer}}, n_{\text{embd}}) = (36, 1280)$ 的模型。一旦测试损失不再下降，我们就停止训练。可以看到，由此得到的测试损失可以用关于数据集大小的简单幂律

$$L(D) \approx \left(\frac{D_c}{D}\right)^{\alpha_D} \quad (3.2)$$

进行拟合。数据和拟合结果如图 1 所示。

训练期间使用的 non-embedding 计算总量可以估计为 $C = 6NBS$ ，其中 B 是 batch size， S 是参数更新次数，而系数 6 用于计入前向传播和反向传播。因此，对于给定的 C 值，我们可以扫描具有不同 N 的所有模型，找出在第 $S = \frac{C}{6BS}$ 步上性能最好的模型。请注意，在这些结果中，所有模型的 batch size B 都保持不变，这意味着这些经验结果并非真正最优。我们将在后面的章节中使用经过调整的 $C_{\min}$ 来考虑这一点，从而得到更清晰的趋势。

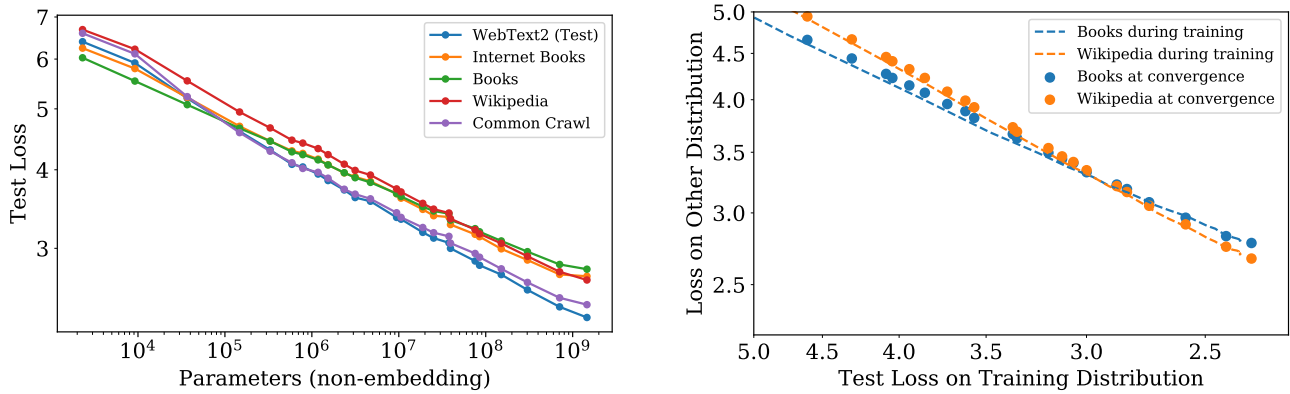


图 8 左图：对其他数据分布的泛化性能随模型规模平滑提升，与 WebText2 训练分布之间只有很小且增长极其缓慢的偏移。右图：泛化性能只取决于训练分布上的性能，而与训练阶段无关。我们将已收敛模型的泛化性能（点）与单个大模型在训练过程中的泛化性能（虚线曲线）进行比较。

结果显示为图 1 左图中的粗黑线。它可以用下式拟合：

$$L(C) \approx \left(\frac{C_c}{C} \right)^{\alpha_C} \quad (3.3)$$

该图还展示了各个模型的学习曲线，用以阐明它们分别在何时最优。我们将在后文更深入地研究计算量的最优分配。数据强烈表明，样本效率随模型规模增大而提高；我们还在附录的图 19 中直接展示了这一点。

4 描绘无限数据极限与过拟合

在第 3 节中，我们发现了语言建模性能的若干基本 scaling laws。这里，我们将研究规模为 N 、在含有 D 个 tokens 的数据集上训练的模型，并同时改变 N 和 D 。我们将通过实验证明，经过最优训练的测试损失符合公式 (1.5) 的 scaling law。这为我们提供了指导：在控制过拟合的同时，训练规模不断增大的模型需要多少数据。

4.1 提议的 $L(N, D)$ 公式

我们选择了参数化形式 (1.5)（为方便起见在此重列）：

$$L(N, D) = \left[\left(\frac{N_c}{N} \right)^{\frac{\alpha_N}{\alpha_D}} + \frac{D_c}{D} \right]^{\alpha_D} \quad (4.1)$$

这基于三个原则：

1. 词表大小或 tokenization 的变化预计会使损失按一个整体因子重新缩放。 $L(N, D)$ 的参数化形式（以及损失的所有模型）必须自然地允许这种重新缩放。
2. 固定 D 并令 $N \rightarrow \infty$ ，整体损失应趋近于 $L(D)$ 。反之，固定 N 并令 $D \rightarrow \infty$ ，损失必须趋近于 $L(N)$ 。
3. $L(N, D)$ 在 $D = \infty$ 处应当是解析的，从而它能够按 $1/D$ 的整数次幂作级数展开。支持这一原则的理论依据明显弱于前两个原则。

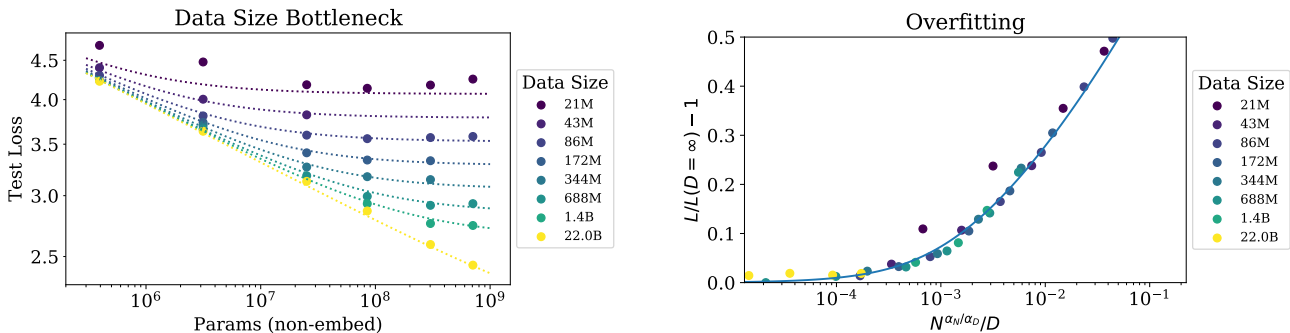


图 9 根据公式 (1.5)，early-stopped 测试损失 $L(N, D)$ 以可预测的方式取决于数据集大小 D 和模型规模 N 。左图：当 D 很大时，性能关于 N 呈严格的幂律关系。当固定的 D 较小时，随着 N 增大，性能会停止改善，模型开始过拟合。（反过来也成立，见图 4。）右图：正如公式 (4.3) 所预测的，过拟合的程度主要取决于比率 $N^{\frac{\alpha_N}{\alpha_D}}/D$ 。曲线是我们对该公式的拟合。

我们选择的 $L(N, D)$ 满足第一项要求，因为词表变化时，我们可以重新缩放 N_c, D_c 。这也意味着 N_c, D_c 的取值没有根本性的意义。

由于我们会在测试损失停止改善时进行 early stopping，并以相同方式优化所有模型，因此我们预计更大的模型应当始终优于更小的模型。但在固定且有限的 D 下，我们也不认为任何模型能够接近可能达到的最佳损失（即文本的熵）。类似地，规模固定的模型将受到容量限制。这些考虑促成了我们的第二个原则。请注意，知道无限 D 时的 $L(N)$ 和无限 N 时的 $L(D)$ ，就能完全确定 $L(N, D)$ 中的所有参数。

第三个原则更具推测性。我们可能预期，在 D 非常大时，过拟合按 $\propto 1/D$ 缩放，这背后有一个简单而普遍的原因。过拟合应与数据集的方差或信噪比有关 [AS17]，而这会按 $1/D$ 缩放。对于任何平滑的损失函数，这一预期都应成立，因为我们预计可以围绕 $D \rightarrow \infty$ 极限展开损失。然而，这一论证假设 $1/D$ 修正相较其他方差来源占主导地位，例如有限的 batch size 以及其他对优化效力的限制。若无实验确认，我们不会对它的适用性抱有很强的信心。

我们的第三个原则解释了公式 (1.5) 中 N 和 D 所扮演角色的不对称性。非常相似的对称表达式³也是可能的，但它们无法按 $1/D$ 的整数次幂展开，并且还需要引入一个额外参数。

无论如何，我们将会看到，关于 $L(N, D)$ 的公式能够很好地拟合数据；这正是我们的 $L(N, D)$ 假设形式最重要的依据。

4.2 结果

我们对所有模型都使用 10% 的 dropout 进行正则化，同时跟踪测试损失，并在其不再下降时停止训练。结果展示在图 9 中，其中还包括对公式 (1.5) 中四个参数 $\alpha_N, \alpha_D, N_c, D_c$ 的拟合：

参数	α_N	α_D	N_c	D_c
取值	0.076	0.103	6.4×10^{13}	1.8×10^{13}

表 2 对 $L(N, D)$ 的拟合

除了数据集缩小了 1024 倍、降至约 2×10^7 个 tokens 的训练运行之外，我们得到了非常出色的拟合。对于如此小的数据集，一个轮次仅包含 40 次参数更新。也许这样一个极小的数据集代表了语言建模的另一种状态，因为过拟合在训练的极早期就会发生（见图 16）。还请注意，这些参数与第 3 节中得到的参数略有不同，因为我们这里我们拟合的是完整的 $L(N, D)$ ，而不只是 $L(N, \infty)$ 或 $L(\infty, D)$ 。

为了描绘无限数据极限的边界地带，我们可以直接研究过拟合的程度。除最大的模型之外，对于所有其他模型，使用完整的、含 22B tokens 的 WebText2 数据集训练时，我们都没有看到过拟合的迹象，因此可以将其视作 $D = \infty$ 的代表。于是，我们可以通过定义

$$\delta L(N, D) \equiv \frac{L(N, D)}{L(N, \infty)} - 1 \quad (4.2)$$

并将其作为 N, D 的函数来研究，从而比较有限 D 与无限数据极限。事实上，如图 16 所示，我们从实验中看到， δL 仅取决于 N 和 D 的一个特定组合。这可以由公式 (1.5) 的 scaling law 推出，该定律意味着

$$\delta L \approx \left(1 + \left(\frac{N}{N_c} \right)^{\frac{\alpha_N}{\alpha_D}} \frac{D_c}{D} \right)^{\alpha_D} - 1 \quad (4.3)$$

请注意，在 D 很大时，该公式同样可以按 $1/D$ 的幂作级数展开。

我们估计，在使用不同随机种子时，损失的变化约为 0.02；这意味着，如果希望训练至距离收敛不超过这一阈值时仍能避免过拟合，我们需要

$$D \gtrsim (5 \times 10^3) N^{0.74} \quad (4.4)$$

按照这一关系，参数量少于 10^9 的模型可以在含 22B tokens 的 WebText2 数据集上训练，并且过拟合程度极小；但我们最大的模型将会遇到一些轻微的过拟合。更一般而言，这一关系表明，在避免过拟合的同时，数据集大小可以相对于模型规模作次线性增长。但请注意，这通常并不代表计算效率最高的训练。我们还应强调，在改变数据集大小和模型规模时，我们并未优化正则化设置（例如 dropout 概率）。

5 模型规模与训练时间的 Scaling Laws

在本节中，我们将证明，一条简单的 scaling law 能够很好地描述损失如何随模型规模 N 和训练时间变化。首先，我们将说明如何利用 [MKAT18] 的结果来定义通用训练步数 $S_{\min}$ ，它考虑到了我们的大多数模型并未在最优 batch size 下训练这一事实。随后，我们将证明，可以使用公式 (1.6) 来拟合损失对模型规模和训练时间的依赖关

³例如，可以使用 $L(N, D) = \left[\left(\frac{N}{N_c} \right)^{\alpha_N} + \left(\frac{D_c}{D} \right)^{\alpha_D} \right]^{\beta}$ ，但它不存在关于 $1/D$ 的展开。

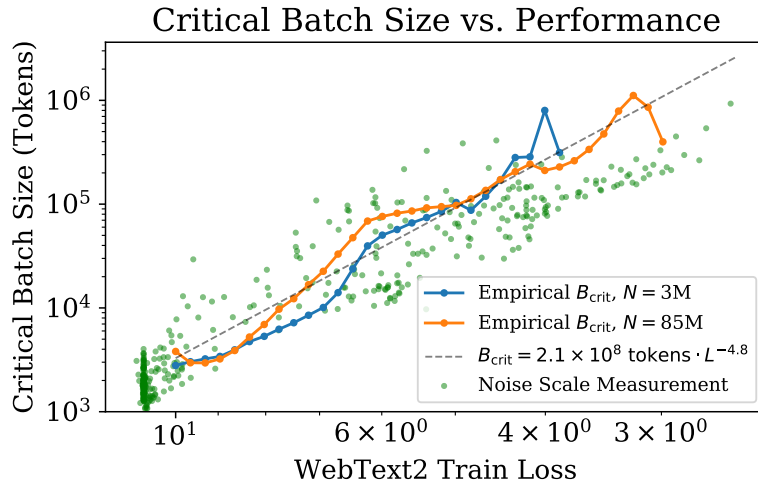


图 10 随着性能提高，critical batch size B_{crit} 关于损失呈幂律关系，并且不直接依赖于模型规模。我们发现，损失每降低 13%，critical batch size 大约翻一番。 B_{crit} 是根据图 18 所示数据通过实验测得的，但正如 [MKAT18] 中那样，gradient noise scale 也能对它作出大致预测。

系。之后，我们将使用这些结果来预测如何在模型规模和训练时间之间最优地分配训练计算量，然后验证这一预测。

5.1 针对在 $B_{\text{crit}}(L)$ 下训练的校正

[MKAT18] 提出了一种关于训练如何依赖 batch size 的简单经验理论（另见 [SLA⁺18, ZLN⁺19]）。文中指出，训练存在一个 critical batch size B_{crit} ；当 B 不超过 B_{crit} 时，可以增大 batch size，而计算效率几乎不会降低；当 $B > B_{\text{crit}}$ 时，增大 B 则会产生收益递减。文中还指出，gradient noise scale 可以对 B_{crit} 作出简单预测，而且二者都不直接依赖于模型规模，只会通过已经达到的损失值间接依赖于它。可以利用这些结果预测训练时间和计算量将如何随 batch size 变化。为了尽可能有效地利用训练时间和计算量，最好使用 batch size $B \approx B_{\text{crit}}$ 进行训练。在 $B \gg B_{\text{crit}}$ 时训练，会使训练步数最少；而在 $B \ll B_{\text{crit}}$ 时训练，则会使计算量最少。

更具体而言，已有研究证明，对于多种多样的神经网络任务，训练步数 S 和已处理的数据样本数 $E = BS$ 满足如下简单关系

$$\left(\frac{S}{S_{\min}} - 1\right) \left(\frac{E}{E_{\min}} - 1\right) = 1 \quad (5.1)$$

该关系适用于训练达到任意固定损失值 L 的情形。这里， $S_{\min}$ 是达到 L 所需的最少步数，而 $E_{\min}$ 是必须处理的最少数据样本数。

我们证明了关系 (5.1) 对 Transformer 同样成立，结果见附录中的图 18。该关系定义了 critical batch size

$$B_{\text{crit}}(L) \equiv \frac{E_{\min}}{S_{\min}} \quad (5.2)$$

它是目标损失值的函数。在 critical batch size 下训练，可在时间与计算量之间作出大致最优的权衡，需要 $2S_{\min}$ 个训练步，并处理 $E = 2E_{\min}$ 个数据样本。

在图 10 中，我们绘制了两个不同模型的 critical batch size 和 gradient noise scale⁴关于训练损失的函数。我们看到， $B_{\text{crit}}(L)$ 与模型规模无关，只取决于损失 L 。因此，[MKAT18] 的预测对 Transformer 语言模型仍然成立。critical batch size 可以用关于损失的幂律来拟合：

$$B_{\text{crit}}(L) \approx \frac{B_*}{L^{1/\alpha_B}} \quad (5.3)$$

其中 $B_* \approx 2 \times 10^8$ ， $\alpha_B \approx 0.21$ 。

我们之所以为 $B_{\text{crit}}(L)$ 选择这一参数化形式，是因为当损失趋近其最小值 $L_{\min}$ 时，gradient noise scale 预计会发散，而我们预计 B_{crit} 会追随这一 gradient noise scale。我们并不知道 $L_{\min}$ ，因为没有看到模型正在趋近它的迹象；但 $L_{\min} > 0$ ，因为自然语言的熵非零。由于 $L_{\min}$ 显然远小于我们已经达到的 L 值，因此我们使用了一种当 $L \rightarrow 0$ 时 B_{crit} 发散的参数化形式。

我们将使用 $B_{\text{crit}}(L)$ 来估计以下两者之间的关系：在 batch size $B = 2^{19}$ tokens 下训练时的训练步数 S ，以及

⁴尽管 critical batch size 与 gradient noise scale 大致吻合，但在之后的所有分析中，我们使用的是根据图 18 和图 10 直接测得的 B_{crit} 。

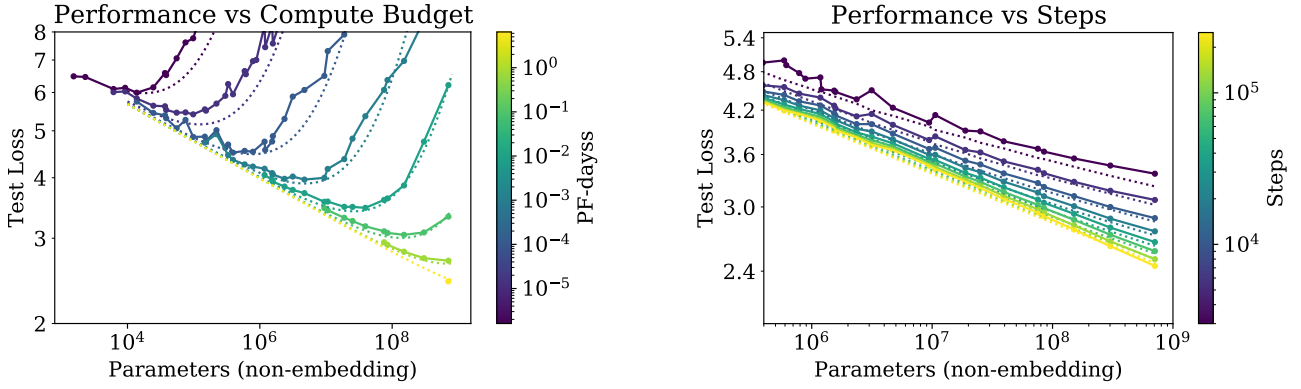


图 11 当我们固定总计算量或训练步数时，性能遵循公式 (5.6) 中的 $L(N, S)$ 。每个计算预算值都有一个使性能最大化的相应最优模型规模。在 S 较小时拟合效果一般并不令人意外，因为学习曲线的幂律公式会在训练极早期失效。

在 $B \gg B_{\text{crit}}$ 下训练时的训练步数。这一关系就是

$$S_{\min}(S) \equiv \frac{S}{1 + B_{\text{crit}}(L)/B} \quad (\text{最少步数, 条件为 } B \gg B_{\text{crit}}) \quad (5.4)$$

其中损失可以取任意给定的目标值 L 。如果我们在 $B \ll B_{\text{crit}}(L)$ 下训练，这也定义了使用规模为 N 的模型训练至 L 所需计算量的临界值。它是

$$C_{\min}(C) \equiv \frac{C}{1 + B/B_{\text{crit}}(L)} \quad (\text{最小计算量, 条件为 } B \ll B_{\text{crit}}) \quad (5.5)$$

其中 $C = 6NBS$ 估计了在 batch size B 下使用的 (non-embedding) 计算量。

5.2 $L(N, S_{\min})$ 的结果以及性能如何随模型规模和计算量变化

现在，我们将使用公式 (5.4) 中定义的 $S_{\min}$ ，对无限数据极限下损失对模型规模和训练时间的依赖关系进行简单而通用的拟合。我们将使用公式 (1.6)（为方便起见在此重列）拟合那些稳定且采用 Adam 优化的训练实验：

$$L(N, S_{\min}) = \left(\frac{N_c}{N}\right)^{\alpha_N} + \left(\frac{S_c}{S_{\min}}\right)^{\alpha_S} \quad (5.6)$$

来表示损失。我们纳入了 learning-rate schedule 预热期之后的所有训练步，并发现数据可用以下参数拟合：

参数	α_N	α_S	N_c	S_c
取值	0.077	0.76	6.5×10^{13}	2.1×10^3

表 3 对 $L(N, S)$ 的拟合

利用这些参数，我们得到了图 4 中的学习曲线拟合。尽管拟合并不完美，但考虑到公式 (5.6) 如此简单，我们认为结果相当有说服力。

如图 11 所示，还可以用一种不同且更有趣的方式将数据和拟合结果可视化。图中，我们研究了在固定训练所用的 non-embedding 总计算量 C 或步数 S 时，测试损失如何随模型规模变化。在拟合中，我们将公式 (5.5) 和 (5.4) 与上述参数以及公式 (5.6) 结合使用。

损失对 $S_{\min}$ 的幂律依赖反映了优化器动力学与损失景观之间的相互作用。由于在训练后期的拟合效果最好，而此时损失可能近似为二次函数，因此该幂律应当能够提供关于损失的 Hessian 矩阵谱的信息。它的普适性表明，Hessian 矩阵的特征值密度大致与模型规模无关。

5.3 Early Stopping 步数的下界

$L(N, S_{\min})$ 的结果可用于推导在数据受限训练中应当进行 early stopping 的步数下界（以及粗略估计）。其动机是：对于给定模型，有限 D 与无限 D 时的学习曲线会非常相似，直到达到 $S_{\min} \approx S_{\text{stop}}$ 。因此，过拟合应当与仅仅在 S_{stop} 时结束训练所带来的修正成正比。这将低估 S_{stop} ，因为现实中，在 D 有限时，测试损失会下降得更慢，因此需要更多训练步才能在有限 D 下达到最优测试损失。这一推理过程导出如下不等式：

$$S_{\text{stop}}(N, D) \gtrsim \frac{S_c}{[L(N, D) - L(N, \infty)]^{1/\alpha_S}} \quad (5.7)$$

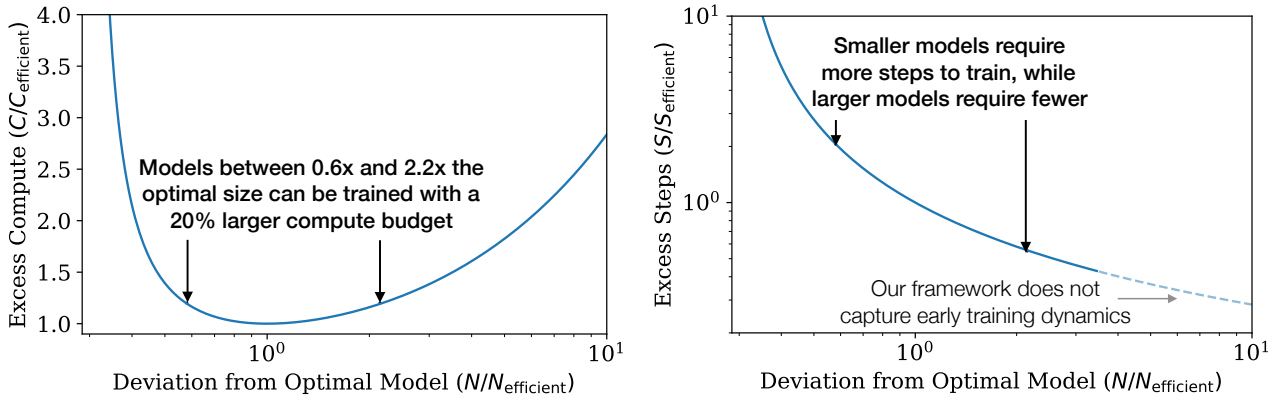


图 12 左图：给定固定的计算预算，某个特定的模型规模是最优的，不过训练略大或略小的模型只需极少的额外计算量。右图：模型规模大于计算高效训练所对应的规模时，所需训练步数更少，因此，如果能够实现足够多的额外并行计算，训练或许可以更快。请注意，对于非常大的模型，不应相信该公式，因为它只在学习曲线的幂律区间内，即初始瞬态效应之后才有效。

其中， $L(N, \infty)$ 是在可用数据无限的条件下评估得到的收敛损失。该不等式及其与实验数据的比较展示在附录的图 16 中。在该图中， S_{stop} 和 $L(N, D)$ 的值来自实验（不过，为了模拟在 $B \gg B_{\text{crit}}$ 下训练，对 S_{stop} 进行了校正），而 $L(N, \infty)$ 是根据对 $L(N, D)$ 的拟合在 $D = \infty$ 处计算得出的。

6 计算预算的最优分配

我们在图 1 的右上方展示了性能如何随训练期间使用的计算量变化这一实验趋势。然而，这一结果涉及在固定 batch size B 下训练，而我们知道，事实上，采用第 5.1 节讨论的 batch size B_{crit} 进行训练可以提高效率⁵。较大和较小的损失值本可分别用更少的样本或更少的步数达到；通过将训练标准化至 critical batch size 来校正这种低效率，会得到更简洁、更可预测的趋势。

在本节中，我们将校正这一疏忽。更重要的是，我们将使用第 5 节的结果，确定如何在模型规模 N 与训练期间处理的数据量（即 $2B_{\text{crit}}S_{\text{min}}$ ）之间最优地分配计算量。我们将使用 $L(N, S_{\text{min}})$ 的公式，以实验和理论两种方式确定这一分配，并证明两种方法得到的结果一致。

6.1 最优性能与分配

首先来研究损失如何随公式 (5.5) 中最优分配的计算量变化。结果绘制在图 13 中，同时给出了幂律拟合。我们看到，与图 1 中的计算量图相比，使用 C_{min} 的新拟合有所改善。

给定 $L(C_{\text{min}})$ ，自然会想知道，在给定训练计算量下，能提供最小损失的最优模型规模 $N(C_{\text{min}})$ 是多少。最优模型规模展示在图 14 中。我们观察到， $N(C_{\text{min}})$ 可以用幂律很好地拟合：

$$N(C_{\text{min}}) \propto (C_{\text{min}})^{0.73}. \quad (6.1)$$

在图 12 中，我们展示了训练非最优规模模型的影响（见附录 B.4）。

⁵人们或许会问，为什么我们一开始不直接在 B_{crit} 下训练。原因在于，它不仅取决于模型，还取决于我们希望达到的目标损失值，因而是一个不断变化的目标。

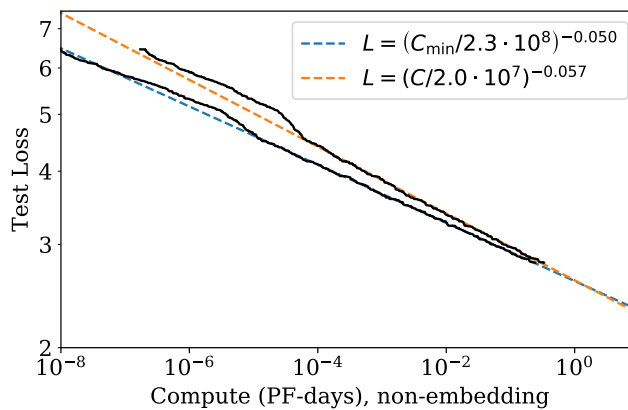


图 13 在对性能进行校正，以模拟远低于 critical batch size 的训练时，我们发现，与完全基于实验的结果相比， $L(C_{\text{min}})$ 的幂律有所改变。位于 10^{-5} PF-日处的显眼凸起标志着从 1 层网络到 2 层网络的转变；我们在幂律拟合中排除了 1 层网络。我们预计， $L(C_{\text{min}})$ 趋势能够为更大计算量提供可靠的外推。

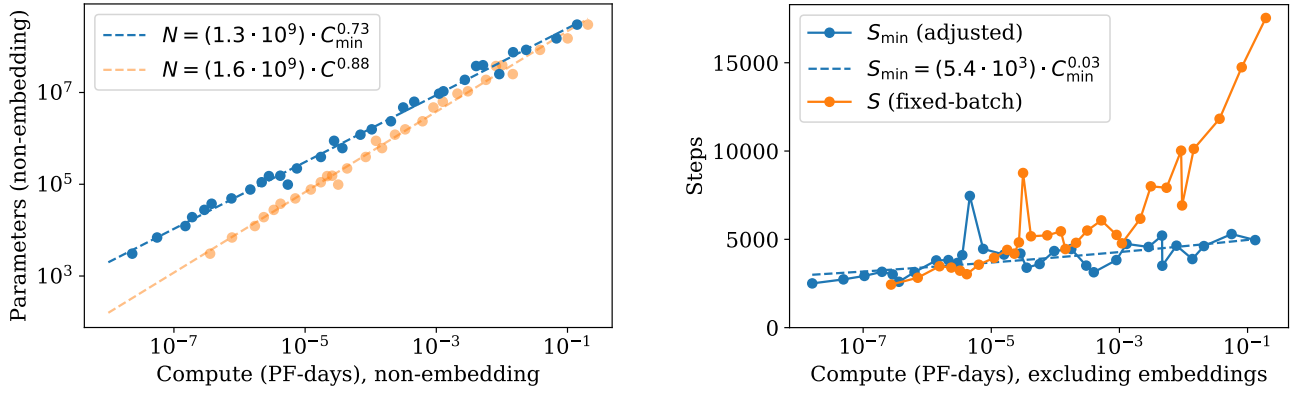


图 14 左图：计算预算 $C_{\min}$ 的每个取值都有一个相应的最优模型规模 N 。最优模型规模随 $C_{\min}$ 极快增长：模型规模每增长 5 倍，对应的计算量增长 10 倍。已处理数据样本数构成其余的增量，增长幅度相对温和，仅为 2 倍。右图：经批量校正的优化步数同样增长得非常缓慢，甚至可能根本没有增长，这意味着已处理数据样本数的大部分增长都可以用于增大 batch size。

根据定义， $C_{\min} \equiv 6NB_{\text{crit}}S$ ，因此可以使用 $N(C_{\min})$ 得出更多结果。特别是，由于先前的拟合表明 $B \propto L^{-4.8}$ 且 $L \propto C_{\min}^{-0.05}$ ，我们可以得出 $B_{\text{crit}} \propto C_{\min}^{0.24}$ 。由此可见，最优步数只会随计算量极其缓慢地增长，即

$$S_{\min} \propto (C_{\min})^{0.03}, \quad (6.2)$$

这与图 14 中的实验结果一致。事实上，测得的指数非常小，以至于我们的结果甚至可能与零指数相符。

因此，我们得出结论：当以最优计算量分配来扩展语言建模时，应当主要增大模型规模 N ，同时通过 $B \propto B_{\text{crit}}$ 增大 batch size，而串行步数几乎无需增加。由于计算高效训练使用的优化步数相对较少，或许有必要进一步研究如何加快训练早期的动力学过程。

6.2 来自 $L(N, S_{\min})$ 的预测

$L(C_{\min})$ 的结果和相应分配可以通过第 5 节所得的 $L(N, S_{\min})$ 公式来预测。给定 $L(N, S_{\min})$ 的公式，我们可以代入 $S_{\min} = \frac{C_{\min}}{6NB}$ ，然后在固定训练计算量的条件下，求损失关于 N 的最小值。我们在附录 B 中详细执行了这一过程，并在其中给出了一些额外预测。

对于损失如何随训练计算量变化，我们预测

$$L(C_{\min}) = \left(\frac{C_{\min}}{C_c} \right)^{\alpha_C^{\min}} \quad (6.3)$$

其中

$$\alpha_C^{\min} \equiv \frac{1}{1/\alpha_S + 1/\alpha_B + 1/\alpha_N} \approx 0.054 \quad (6.4)$$

这与图 13 中的指数高度吻合。我们还预测

$$N(C_{\min}) \propto (C_{\min})^{\alpha_C^{\min}/\alpha_N} \approx (C_{\min})^{0.71} \quad (6.5)$$

这同样在百分之几的范围内与图 14 的缩放关系相符。我们的 scaling laws 为语言建模的性能提供了一个预测框架。

6.3 矛盾与一个猜想

在计算量、数据或模型规模取较大值时，我们没有观察到偏离直线型幂律趋势的迹象。然而，我们的趋势最终必定趋于平缓，因为自然语言的熵非零。

事实上，本节所述计算高效训练的趋势中已经包含了一个表面上的矛盾。当规模比本文记录的高出数个数量级时， $L(C_{\min})$ scaling law 预测的性能会下降到这样一种水平之下：考虑到训练数据随计算量增长得如此缓慢，这一水平本不可能达到。这意味着我们的 scaling laws 必定会在此之前失效，但我们猜想，该交点具有更深层的含义：它给出了 Transformer 语言模型达到最高性能之处的估计。

由于计算高效训练使用的数据量随计算预算缓慢增长， $L(C_{\min})$ 预测的性能最终会触及由 $L(D)$ 幂律设定的下界（见图 15）。下面对此作更详细的推导。

为了控制过拟合，第 4 节的结果表明，我们应按如下方式扩展数据集大小：

$$D \propto N^{0.74} \propto C_{\min}^{0.54} \quad (6.6)$$

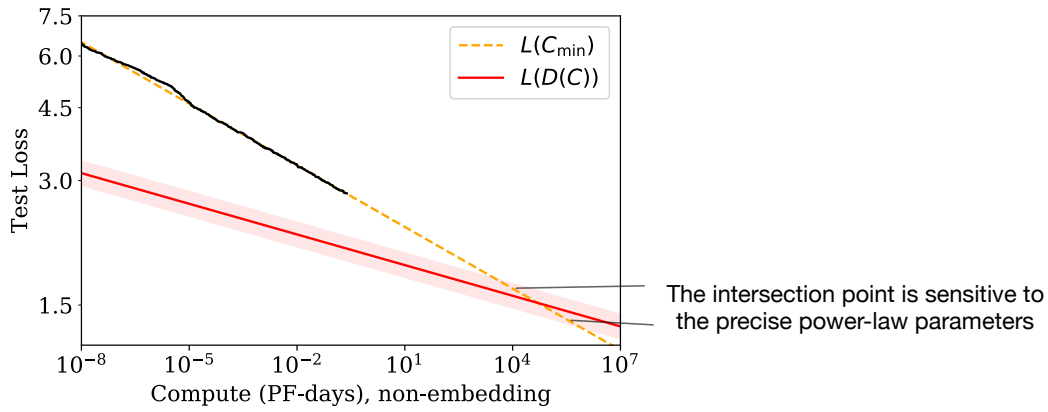


图 15 在远远超出我们通过实验研究的模型规模时，我们发现，由于计算高效训练所需数据增长缓慢，关于 $L(C_{\min})$ 和 $L(D)$ 的公式之间出现了矛盾。交点标出了我们预计预测会在其之前失效的位置。该位置对幂律拟合所得的精确指数极为敏感。

这里，我们使用了图 14 中计算高效训练下的 $N(C_{\min})$ 。

下面将其与计算高效训练的数据需求进行比较。如果在 critical batch size 下训练（即 $C = 2C_{\min}$ ），并且训练期间从不重复使用数据，那么数据使用量随计算量的增长关系为

$$D(C_{\min}) = \frac{2C_{\min}}{6N(C_{\min})} \approx (4 \times 10^{10} \text{ tokens}) (C_{\min}/\text{PF-日})^{0.26} \quad (6.7)$$

这是数据集大小能够随计算量有效增长的最大速率，因为这意味着我们只训练一个轮次。但与公式 (6.6) 相比，它使数据集增长得慢得多。它似乎意味着，即使训练过程从不重复使用任何数据，计算高效训练最终也会遇到过拟合问题！

根据图 1，我们预计，当瓶颈在于数据集大小（即受过拟合限制）时，损失应按 $L(D) \propto D^{-0.095}$ 缩放。这意味着，一旦受数据限制，损失将随计算量按 $L(D(C_{\min})) \propto C_{\min}^{-0.03}$ 缩放。我们再一次遇到了矛盾，因为这最终会与图 13 中关于 $L(C_{\min})$ 的预测相交；在该图中，我们发现的缩放关系是 $L(C_{\min}) \propto C_{\min}^{-0.050}$ 。

$L(D(C_{\min}))$ 与 $L(C_{\min})$ 的交点位于

$$C^* \sim 10^4 \text{ PF-日} \quad N^* \sim 10^{12} \text{ 个参数}, \quad D^* \sim 10^{12} \text{ tokens}, \quad L^* \sim 1.7 \text{ 纳特/token} \quad (6.8)$$

不过，这些数值具有很大的不确定性：取决于幂律拟合所得指数的精确取值，它们可能向任一方向变化一个数量级。最显而易见的解释是，我们的 scaling laws 会在达到这一点时或此前失效；而无论就计算量还是模型规模而言，这一点仍相距许多个数量级。

也可以猜想，该交点具有更深层的含义。如果不在数据需求上发生质变，我们就无法将模型规模增大到 N^* 以上，那么也许这意味着，一旦达到 $C_{\min}^*$ 和 N^* ，我们就已经提取了自然语言数据中所有可靠的信息。在这一解释下， L^* 将为自然语言的每 token 熵⁶提供一个粗略估计。在这种情形下，我们预计损失趋势会在 L^* 或此前趋于平缓。

我们可以考虑在训练数据集中加入噪声的版本，以猜测 $L(C_{\min})$ 趋于平缓时的函数形式。例如，可以在展示给模型的每段上下文后附加一个随机 token 串，从而人为地将损失提高一个恒定的加性因子。这样，与噪声下限的距离 $L - L_{\text{noise}}$ 将成为更有意义的性能指标；即使该距离略有减小，也可能代表定性性能的显著提升。由于人为噪声会同等影响我们的所有趋势，6.8 的临界点不会改变（除了 L^* 的绝对值），即使它出现在趋势趋于平缓之后，也可能仍具有意义。

7 相关工作

幂律可以由多种多样的来源产生 [THK18]。密度估计 [Was06] 和随机森林模型 [Bia12] 中随模型和数据集大小变化的幂律缩放关系可能与我们的结果有关。这些模型表明，可以非常粗略地将幂律指数解释为数据中相关特征数量的倒数。

一些早期工作 [BB01, Goo01] 发现了性能与数据集大小之间的幂律缩放关系。较新的工作 [HNA⁺17, HAD19] 还研究了模型规模与数据大小之间的缩放关系；在现有文献中，他们的工作或许与我们最为接近⁷。但请注意，[HNA⁺17] 发现数据集大小相对于模型规模呈超线性缩放，而我们发现的是次线性缩放。我们关于计算量最优分配的发现与 [Kom19] 之间有一些相似之处，其中包括幂律学习曲线。EfficientNets [TL19] 似乎也遵循准确率与模型规模之间的近似幂律关系。最近的一项工作 [RRBS19b] 研究了多种数据集中数据集大小和模型规模同时变化时的缩放关系，并拟合了一种与我们类似的假设形式。

⁶使用 wc 工具定义单词时，WebText2 数据集每个单词含 1.4 tokens，每个 token 含 4.3 个字符。

⁷在本研究完成后，[RRBS19a] 也发表了，该研究对损失如何同时依赖于模型规模和数据集大小作出了类似预测。

EfficientNet [TL19] 主张，为获得图像模型的最优性能，应以指数方式（采用不同系数）扩展深度和宽度，从而使宽度关于深度呈幂律缩放关系。我们发现，对于语言模型，在扩大规模时，该幂指数应当大致为一（因为宽度/深度应保持不变）。但更重要的是，我们发现，相较语言模型的整体规模，具体的架构超参数并不重要。[VWB16] 认为，深层模型可以充当较浅模型的集成，这可能解释这一发现。更早的工作 [ZK16] 比较了宽度和深度，并发现宽 ResNets 在图像分类上的表现可以优于深 ResNets。一些研究固定了每个数据样本的计算量，这一计算量往往与模型参数的数量成比例缩放；而我们研究的是模型规模和训练计算量同时变化时的缩放关系。

多项工作 [AS17, BHMM18] 研究了高度过参数化模型的泛化，发现在模型规模达到数据集大小时会出现一种“堵塞转变” [GJS⁺19]（这可能需要比典型实践多出许多数量级的训练，尤其是不能使用 early stopping）。我们没有观察到这样的转变，并发现所需训练数据相对于模型规模呈次线性缩放。关于模型规模的展开，尤其是宽度很大时的展开 [JGH18, LXS⁺19]，可能为思考我们的某些缩放关系提供一个有用框架。我们关于优化的结果，例如学习曲线的形状，很可能可以用含噪二次模型来解释；在现实情境中，这种模型能够作出相当准确的预测 [ZLN⁺19]。要定量建立这一联系，需要刻画 Hessian 矩阵的谱 [Pap18, GKX19, GARD18]。

8 讨论

我们观察到，语言模型的对数似然损失随 non-embedding parameters 的数量 N 、数据集大小 D 和优化后的训练计算量 $C_{\min}$ 呈现出一致的缩放关系，如公式 (1.5) 和 (1.6) 所概括。相反，我们发现它对许多架构和优化超参数的依赖非常弱。由于随 $N, D, C_{\min}$ 的缩放关系为幂律，随着规模增加，收益会递减。

当 N 与 D 同时变化，或者 N 与 S 同时变化时，我们能够精确地建立损失对它们的依赖模型。我们使用这些关系推导了训练大型语言模型时的计算量缩放、过拟合程度、early stopping 步数和数据需求。因此，我们的缩放关系不止是单纯的观察，还提供了一个预测框架。可以将这些关系理解为理想气体定律的类似物；该定律以普适的方式联系气体的宏观性质，而与其微观组成成分的大多数细节无关。

很自然可以猜想，这些缩放关系将适用于其他采用最大似然损失的生成建模任务，也可能适用于其他情境。为此，在图像、音频和视频模型等其他领域，或许也在随机网络蒸馏中检验这些关系，将会很有意义。目前，我们尚不清楚哪些结果取决于自然语言数据的结构，哪些结果具有普适性。如果能找到一个可以从推导缩放关系的理论框架，也会令人振奋：即为我们所观察到的“热力学”奠定基础的“统计力学”。这样的理论可能使我们能够推导出其他更精确的预测，并系统地理解 scaling laws 的局限。

在自然语言领域，研究损失的持续改善是否会转化为相关语言任务上的改善十分重要。平滑的量变可能掩盖重大的质变：“多则异”。例如，经济总量的平滑增长并不能揭示支撑这种增长的具体技术进步。类似地，语言模型损失的平滑改善可能隐藏着看似定性的能力变化。

我们的结果强烈表明，更大的模型会继续表现得更好，而且样本效率也会比此前认识的更高。大模型可能比大数据更重要。在这种背景下，有必要进一步研究模型并行。深层模型可以使用流水线方法训练 [HCC⁺18]，该方法沿深度方向将参数分配到不同设备之间，但随着使用的设备增多，最终会要求增大 batch size。另一方面，宽网络更适合并行化 [SCP⁺18]，因为大层可以被分割到多个工作节点之间，并且串行依赖更少。稀疏性 [CGRS19, GRK17] 或分支（例如 [KSH12]）可能通过提高模型并行度，实现对大型网络更快的训练。而采用 [WRH17, WYL19] 一类在训练过程中扩展网络的方法，或许可以在整个训练运行期间始终处于计算高效前沿。

致谢

我们感谢 Shan Carter、Paul Christiano、Jack Clark、Ajeya Cotra、Ethan Dyer、Jason Eisner、Danny Hernandez、Jacob Hilton、Brice Menard、Chris Olah 和 Ilya Sutskever 参与讨论，并对本研究的草稿提出反馈。

A 幂律总结

为便于查阅，下面汇总了全文所述的关键趋势。

参数	数据	计算量	Batch Size	方程
N	∞	∞	固定	$L(N) = (N_c/N)^{\alpha_N}$
∞	D	Early Stopping	固定	$L(D) = (D_c/D)^{\alpha_D}$
最优	∞	C	固定	$L(C) = (C_c/C)^{\alpha_C}$ (朴素)
N_{opt}	D_{opt}	$C_{\min}$	$B \ll B_{\text{crit}}$	$L(C_{\min}) = (C_c^{\min}/C_{\min})^{\alpha_C^{\min}}$
N	D	Early Stopping	固定	$L(N, D) = \left[\left(\frac{N_c}{N} \right)^{\frac{\alpha_N}{\alpha_D}} + \frac{D_c}{D} \right]^{\alpha_D}$
N	∞	S 步	B	$L(N, S) = \left(\frac{N_c}{N} \right)^{\alpha_N} + \left(\frac{S_c}{S_{\min}(S, B)} \right)^{\alpha_S}$

表 4

这些趋势的经验拟合值为：

幂律	尺度（依赖于 tokenization）
$\alpha_N = 0.076$	$N_c = 8.8 \times 10^{13}$ non-embedding parameters
$\alpha_D = 0.095$	$D_c = 5.4 \times 10^{13}$ tokens
$\alpha_C = 0.057$	$C_c = 1.6 \times 10^7$ PF-日
$\alpha_C^{\min} = 0.050$	$C_c^{\min} = 3.1 \times 10^8$ PF-日
$\alpha_B = 0.21$	$B_* = 2.1 \times 10^8$ tokens
$\alpha_S = 0.76$	$S_c = 2.1 \times 10^3$ 步

表 5

计算高效训练的最优参数如下：

计算高效值	幂律	尺度
$N_{\text{opt}} = N_e \cdot C_{\min}^{p_N}$	$p_N = 0.73$	$N_e = 1.3 \cdot 10^9$ 个参数
$B \ll B_{\text{crit}} = \frac{B_*}{L^{1/\alpha_B}} = B_e C_{\min}^{p_B}$	$p_B = 0.24$	$B_e = 2.0 \cdot 10^6$ tokens
$S_{\min} = S_e \cdot C_{\min}^{p_S}$ (下界)	$p_S = 0.03$	$S_e = 5.4 \cdot 10^3$ 步
$D_{\text{opt}} = D_e \cdot C_{\min}^{p_D}$ (1 个轮次)	$p_D = 0.27$	$D_e = 2 \cdot 10^{10}$ tokens

表 6

B 计算高效前沿的经验模型

在本附录中， C, S 和 α_C 的所有取值都已针对在 critical batch size B_{crit} 下的训练进行了调整。为避免符号过于繁杂，我们省略了 ‘adj’ 标记。

B.1 定义方程

对学习曲线进行幂律拟合，可以得到一种简单的计算高效训练方案。在本附录中，我们将推导最优性能、模型规模和训练步数与计算预算之间的函数关系。我们从方程 (1.6) 开始，为方便起见在此重复如下：

$$L(N, S) = \left(\frac{N_c}{N} \right)^{\alpha_N} + \left(\frac{S_c}{S} \right)^{\alpha_S}. \quad (\text{B.1})$$

这里， S 表示在 critical batch size 下训练时的参数更新次数 [MKAT18]，critical batch size 在方程 (5.2)⁸ 中定义：

$$B(L) = \frac{B_*}{L^{1/\alpha_B}}. \quad (\text{B.2})$$

⁸这里存在一点歧义：我们可以设想在恒定 batch size $B(L_{\text{target}})$ 下训练，也可以改为在可变 batch size $\tilde{B}(L)$ 下训练，其中 $\tilde{B}$ 是瞬时 critical batch size（相对于 B ，后者是平均版本）。这两种方案产生的步数相同，因此我们可以忽略这一细微差别（见 [MKAT18]）。

我们希望确定固定计算预算下的最优训练参数，因此作替换 $S = C / (6NB(L))$ ，其中 C 是该次训练所使用的 FLOP 数：

$$L(N, C) = \left(\frac{N_c}{N}\right)^{\alpha_N} + \left(6B_* S_c \frac{N}{L^{1/\alpha_B} C}\right)^{\alpha_S}. \quad (\text{B.3})$$

现在，我们令 $\partial_N L|_C = 0$ ，以求得最优条件：

$$\begin{aligned} 0 &= \frac{\partial L}{\partial N}|_C \\ &= -\frac{\alpha_N}{N} \left(\frac{N_c}{N}\right)^{\alpha_N} + \frac{\alpha_S}{N} \left(6B_* S_c \frac{N}{L^{1/\alpha_B} C}\right)^{\alpha_S} \left(1 - 5 \frac{N}{L} \frac{\partial L}{\partial N}|_C\right) \\ \Rightarrow \frac{\alpha_N}{\alpha_S} \left(\frac{N_c}{N}\right)^{\alpha_N} &= \left(6B_* S_c \frac{N}{L^{1/\alpha_B} C}\right)^{\alpha_S} \end{aligned} \quad (\text{B.4})$$

方程 (B.3) 和 (B.4) 共同确定了计算高效前沿。

B.2 高效训练

现在，我们汇总 (B.3) 和 (B.4) 的推论。首先，注意到将 (B.4) 代入 (B.3) 可得

$$L(N_{\text{eff}}(C), C) = \left(1 + \frac{\alpha_N}{\alpha_S}\right) L(N_{\text{eff}}, \infty), \quad (\text{B.5})$$

这意味着，对于计算高效训练，我们应训练到比收敛损失高出**固定百分比** $\frac{\alpha_N}{\alpha_S} \approx 10\%$ 的水平。接下来，我们确定最优损失如何依赖于计算预算。消去 N ，可得到性能关于计算量的幂律依赖关系：

$$L(C) = \left(\frac{C_c}{C}\right)^{\alpha_C} \quad (\text{B.6})$$

其中我们定义

$$\alpha_C = 1 / (1/\alpha_S + 1/\alpha_B + 1/\alpha_N) \approx 0.052 \quad (\text{B.7})$$

$$C_c = 6N_c B_* S_c \left(1 + \frac{\alpha_N}{\alpha_S}\right)^{1/\alpha_S + 1/\alpha_N} \left(\frac{\alpha_S}{\alpha_N}\right)^{1/\alpha_S}. \quad (\text{B.8})$$

类似地，我们可以消去 L ，求得 $N(C)$ ：

$$\frac{N(C)}{N_c} = \left(\frac{C}{C_c}\right)^{\alpha_C/\alpha_N} \left(1 + \frac{\alpha_N}{\alpha_S}\right)^{1/\alpha_N} \quad (\text{B.9})$$

以及

$$S(C) = \frac{C_c}{6N_c B_*} \left(1 + \frac{\alpha_N}{\alpha_S}\right)^{-1/\alpha_N} \left(\frac{C}{C_c}\right)^{\alpha_C/\alpha_S} \quad (\text{B.10})$$

B.3 与低效训练的比较

通常，研究人员会训练模型，直到它们看起来接近收敛。在本节中，我们将上述高效训练过程与这种更为典型的设置进行比较。我们将收敛因子 f 定义为相对于收敛损失的百分比偏差：

$$L(N, C) = (1 + f) L(N, \infty). \quad (\text{B.11})$$

由上一节可知，对于计算高效训练， $f = \alpha_N/\alpha_S \approx 10\%$ ，但研究人员通常使用小得多的值。这里，我们选择 $f' = 2\%$ 作为估计值。对于固定的损失值，我们预测：

$$\frac{N_f}{N_{f'}} = \left(\frac{1+f}{1+f'}\right)^{1/\alpha_N} \approx 2.7 \quad (\text{B.12})$$

$$\frac{S_f}{S_{f'}} = \left(\frac{1+\frac{1}{f}}{1+\frac{1}{f'}}\right)^{1/\alpha_S} \approx 0.13 \quad (\text{B.13})$$

$$\frac{C_f}{C_{f'}} = \frac{N_f}{N_{f'}} \frac{S_f}{S_{f'}} \approx 0.35 \quad (\text{B.14})$$

因此，为达到相同的损失，计算高效训练所需的参数更新次数缩减 7.7 倍、参数数量增至 2.7 倍，并少用 65% 的计算量。

B.4 次优模型规模

我们可以求解 A.1，得到达到给定损失值 L 时，规模为 N 的模型所需计算量的表达式：

$$C(N, L) = \left(6B_* S_c \frac{N}{L^{1/\alpha_B}} \right) \left(L - \left(\frac{N_c}{N} \right)^{\alpha_N} \right)^{-1/\alpha_S}. \quad (\text{B.15})$$

利用 A.6 和 A.9，我们可以消去 L ，改用 $N_{\text{eff}}(L)$ 表示，其中后者是以最高效率达到 L 的模型规模。由此，我们得到因使用次优模型规模而需要的额外计算量的表达式：

$$\frac{C(N, N_{\text{eff}})}{C(N_{\text{eff}}, N_{\text{eff}})} = \frac{N}{N_{\text{eff}}} \left[1 + \frac{\alpha_S}{\alpha_N} \left(1 - \left(\frac{N_{\text{eff}}}{N} \right)^{\alpha_N} \right) \right]^{-1/\alpha_S}. \quad (\text{B.16})$$

结果如图 X 所示。可以使用规模介于最优规模 0.6 倍和 2.2 倍之间的模型，而计算预算仅增加 20%。考虑 inference 成本时，使用较小的模型很有价值。更大的模型可以用更少的步数训练到相同的性能水平，从而在硬件充足时实现更高的并行度和更快的训练（见图 Y）：

$$\frac{S(N, N_{\text{eff}})}{S(N_{\text{eff}}, N_{\text{eff}})} = \left[1 + \frac{\alpha_S}{\alpha_N} \left(1 - \left(\frac{N_{\text{eff}}}{N} \right)^{\alpha_N} \right) \right]^{-1/\alpha_S}. \quad (\text{B.17})$$

规模为 2.2 倍的模型所需步数减少 45%，代价是训练计算量增加 20%。请注意，对于非常大的模型，不应信赖这个方程，因为它仅在学习曲线经过初始瞬态效应之后的幂律区域内有效。

C 注意事项

本节列出了我们分析中一些潜在的注意事项。

- 目前，对于我们提出的所有 scaling laws，我们都缺乏扎实的理论理解。模型规模和计算量的缩放关系尤其难以解释。通过用带噪二次函数对损失建模，或许可以理解在模型规模固定、 D 非常大时的缩放关系 [AS17]，以及训练后期学习曲线的形状。但在模型规模非常大时随 D 的缩放关系仍然难以解释。如果没有理论，或无法系统理解对 scaling laws 的修正，就很难确定在什么情况下可以信赖它们。
- 对于远在我们所探索范围之外的损失值，我们并不特别确信对 $B_{\text{crit}}(L)$ 的预测。 B_{crit} 的变化可能会显著影响数据并行度与所需串行训练步数之间的权衡，进而对训练时间产生重大影响。
- 我们没有彻底研究小数据情形，并且我们对 $L(N, D)$ 的拟合在最小的 D 值时效果较差（此时一个轮次仅对应 40 步）。此外，我们没有试验正则化和数据增强。对这些方法的改进可能会从定量或定性上改变我们的结果。
- 我们使用了估算的训练计算量 $C \approx 6NBS$ ，其中没有包含与 n_{ctx} 成正比的贡献（见第 2.1 节）。因此，在 n_{ctx} 非常大的区域，特别是 $n_{\text{ctx}} \gtrsim 12d_{\text{model}}$ 时，我们关于计算量的缩放关系在实践中可能会受到混杂因素的影响。
- 我们调节了 learning rate，并试验了 learning-rate schedule。但我们可能忽略了对某些会显著影响缩放的超参数（例如初始化尺度或动量）进行调节。
- learning rate 的最优选择对目标损失很敏感。当训练接近收敛时，可能需要使用较小的 learning rate 来避免发散。但在进行短时间训练时（例如由于计算量限制），或许可以使用更大的 learning rate。对于未进行到收敛的训练，我们没有试验更高的 learning rate。

D 补充图

D.1 Early Stopping 以及测试与训练的比较

在第 5.3 节中，我们描述了图 16 所示的结果，该结果预测了 early stopping 步数的下界。我们还展示了在不同规模的数据集上训练时，给定模型规模对应的训练损失和测试损失。

D.2 通用 Transformer

我们比较了标准 Transformer 与循环 Transformer [DGV⁺18] 的性能，结果见图 17。这些模型会复用参数，因此按 N 衡量时表现略好，但按计算量 C 衡量时表现略差。我们纳入了几种不同的参数复用方式。

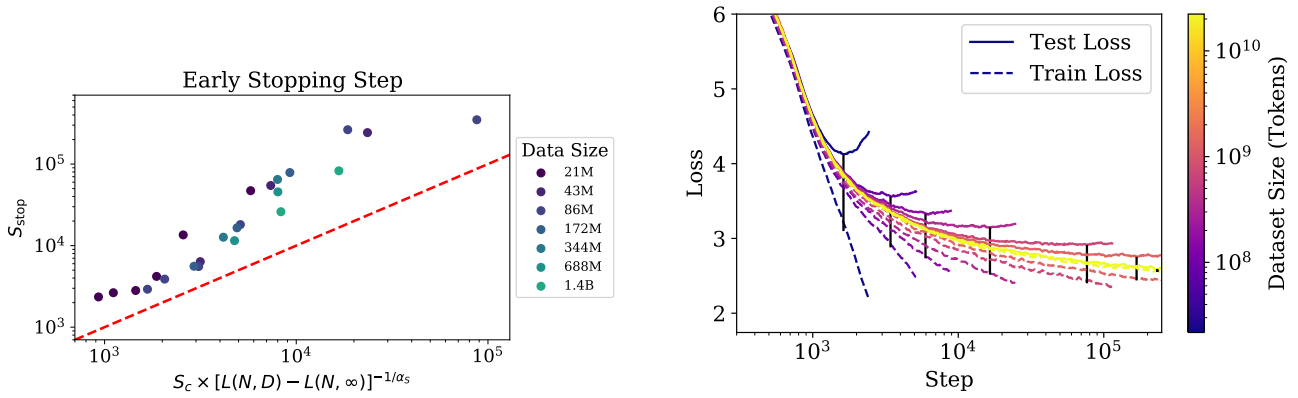


图 16 左：我们刻画了 early stopping 发生的步数与过拟合程度之间的函数关系。红线表示第 5.3 节中推导出的 early stopping 下界。右：我们展示了一系列 3 亿参数模型的训练损失和测试损失，这些模型在不同规模的数据集子样本上训练。测试损失通常会跟随使用无限制数据的训练运行，直到两者出现偏离。请注意， $L_{\text{test}} - L_{\text{train}}$ 会显著高估过拟合程度（与无限数据极限相比），图中每次训练都用黑色条标出了这一差值。

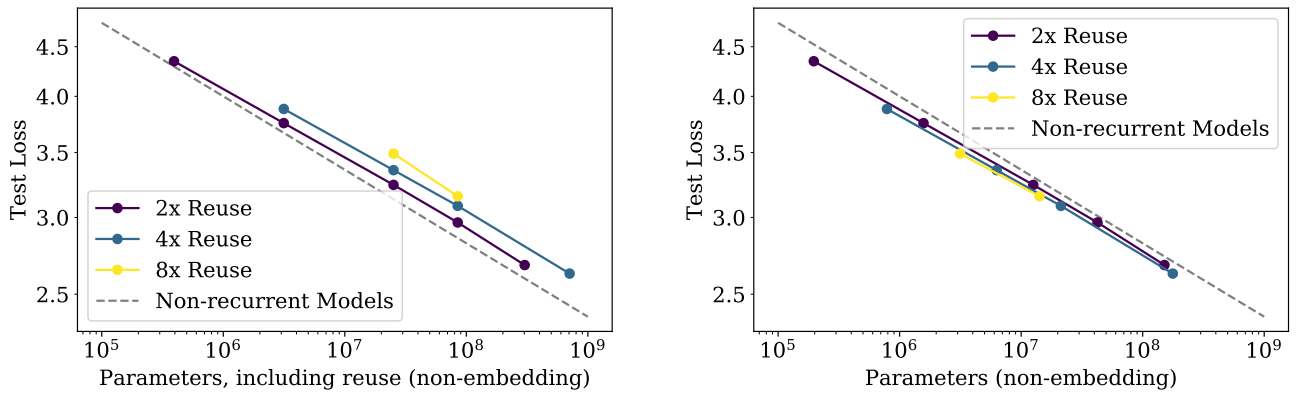


图 17 我们将复用参数的循环 Transformer [DGV⁺18] 与标准 Transformer 进行比较。在比较参数量相同的模型时，循环 Transformer 的表现略好；但考虑参数复用并按每 FLOP 比较时，其表现略差。

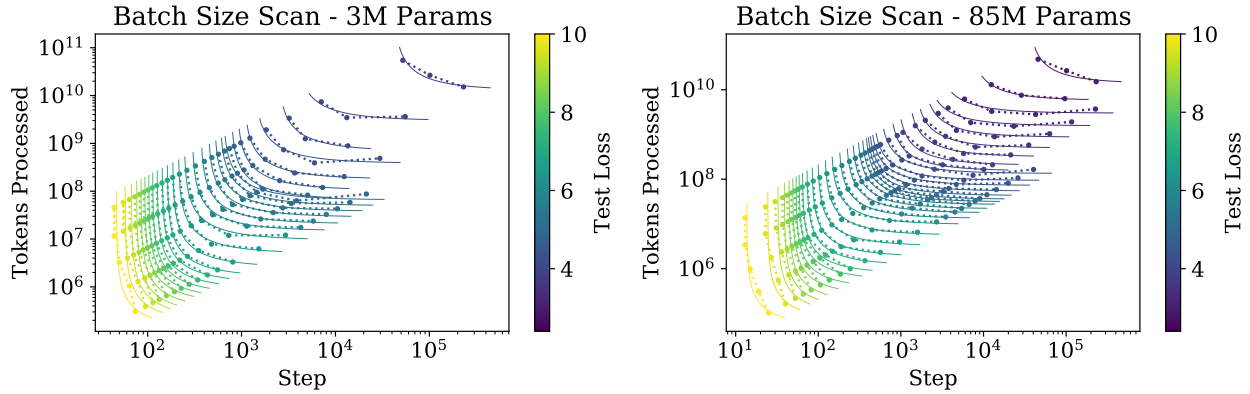


图 18 这些图展示了在损失 L 的大量不同取值下，针对两种不同规模的 Transformer 模型对方程 (5.1) 所做的拟合。这些拟合用于测量 $B_{\text{crit}}(L)$ ，以绘制图 10。

D.3 Batch Size

我们使用图 18 所示的数据测量 critical batch size。由此可以估算 $B_{\text{crit}}(L)$ ，如图 10 所示。

D.4 样本效率与模型规模

从图 2 中很容易看出，较大的模型训练得更快，因此样本效率也更高。我们在图 19 中提供了观察这一现象的另一种方式，该图展示了不同模型何时达到各个固定的损失值。

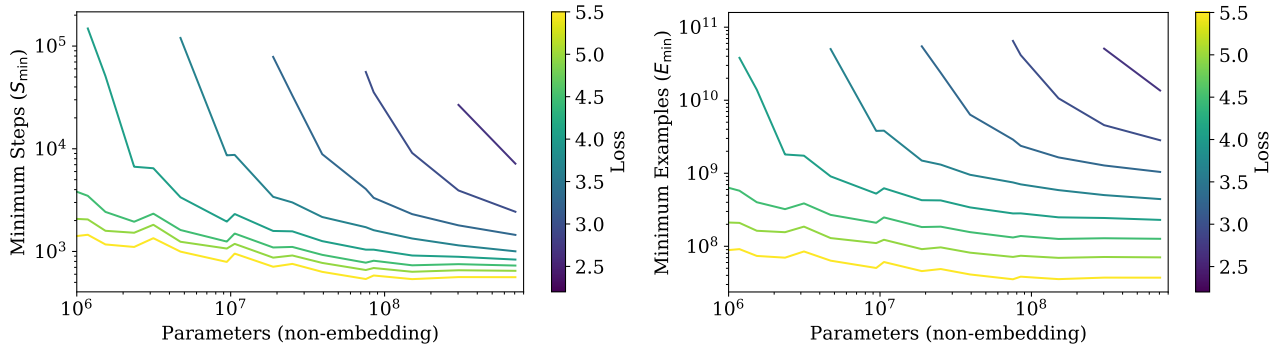


图 19 达到任意固定测试损失值所需的最少串行步数会随模型规模增大而急剧减少。样本效率（这里展示的是远低于 critical batch size 的训练）也有大幅提高；将最小的可行模型与非常大的模型进行比较时，提升幅度接近 100 倍。

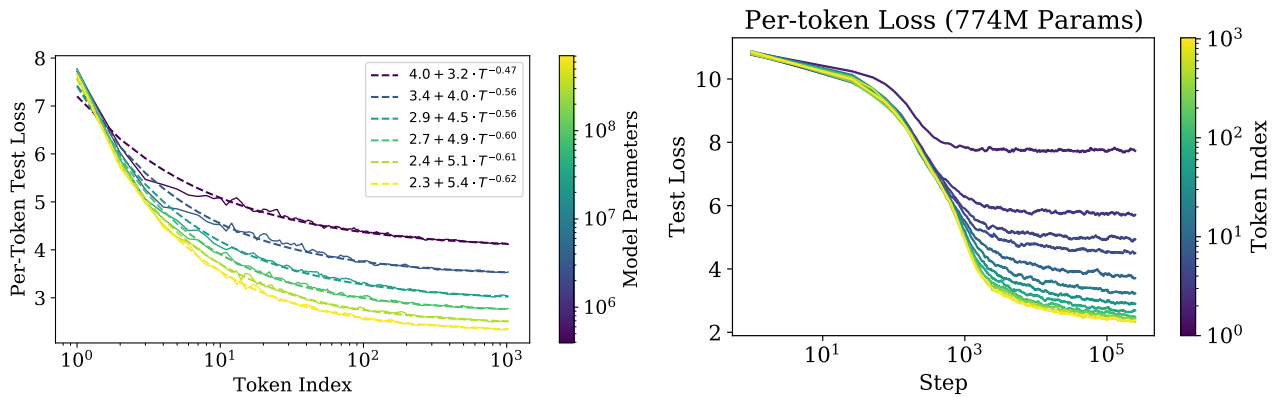


图 20 此图提供了每个 token 的性能如何随模型规模和训练时间变化的信息。左：每个 token 的损失与其在 1024-token 上下文中的位置 T 之间的函数关系。损失以可预测的方式随 T 呈幂律缩放。右：每个 token 的测试损失与训练步数之间的函数关系。

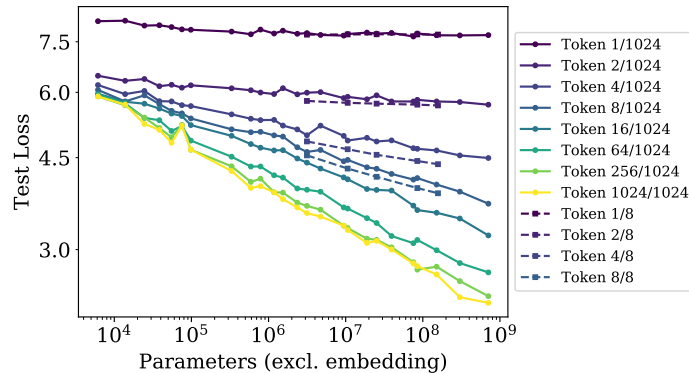


图 21 除了平均损失外，1024-token 上下文中的各个 token 也会随着模型规模增大而平稳改善。使用较短上下文 $n_{\text{ctx}} = 8$ 训练的模型（虚线）在较早的 token 上表现更好，因为它们可以将全部容量分配给这些 token。

D.5 上下文依赖性

图 21 展示了上下文中不同 token 对应的损失随模型规模变化的趋势。我们看到，在 $n_{\text{ctx}} = 1024$ 上训练的模型，除第一个 token 外，其他所有 token 上的性能都会随模型规模增大而稳步改善。

固定模型规模后，损失似乎会随上下文中的位置 T 呈幂律缩放，见图 20。这可能是语言中底层幂律相关性的结果 [EP94, ACDE12, LT16]，也可能是模型架构和优化更普遍的一项特征。这为使用更大上下文训练可能带来的益处（或缺乏益处）提供了一些提示。较大的模型不仅在 $T = 1024$ 处收敛到更好的性能，而且在较早的 token 上改进得也更快，这表明较大的模型能以更少的上下文信息更高效地检测模式。在右图中，我们展示了固定模型的 per-token 性能如何随训练步数变化。模型首先开始学习短程信息，直到训练后期才会学习长程相关性。

我们还纳入了使用极小上下文 $n_{\text{ctx}} = 8$ 训练的模型，以便与较长上下文模型进行比较。即使是规模适中的、在 $n_{\text{ctx}} = 8$ 上训练的模型，在非常早期的 token 上也能胜过我们最大的 $n_{\text{ctx}} = 1024$ 模型。这也表明，使用长上下文训练的、规模大得多的模型，应该还能取得进一步改进。

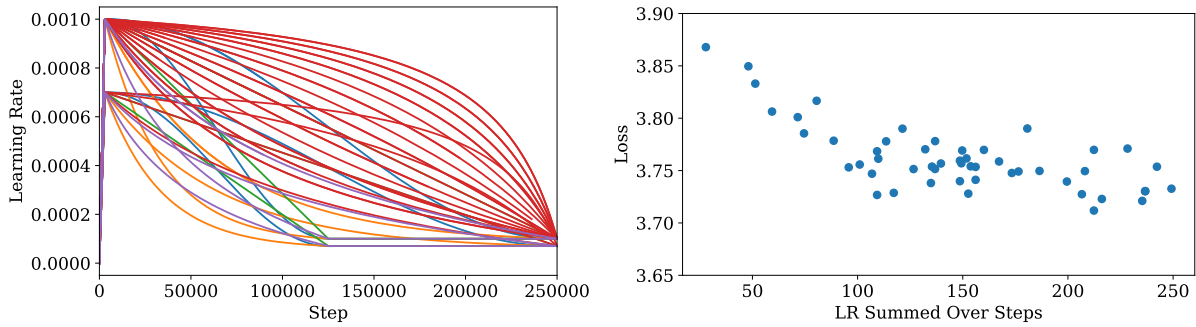


图 22 我们在一个 300 万参数模型上测试了多种 learning-rate schedule，包括余弦衰减、线性衰减，以及其他更快或更慢的衰减调度，如左图所示。在这些实验中，我们没有将 learning rate 衰减到零，因为我们发现，这往往会在接近训练结束时带来一个固定的改进。我们发现，只要 learning rate 不是过小，并且衰减速度不是过快，性能就不会强烈依赖于 learning rate。不同训练运行之间的损失变化约为 0.05，因此，要验证小于这一幅度的性能变化，就必须对多次训练运行取平均。

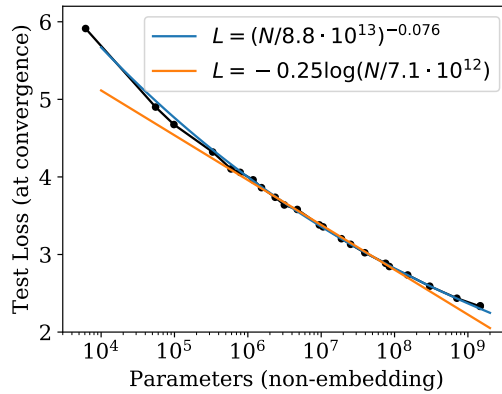


图 23 在定性层面上，以参数量为自变量的性能趋势 $L(N)$ 用幂律拟合比用对数等其他函数拟合得更好。

D.6 learning-rate schedule 与误差分析

我们试验了多种 learning rate 和调度。图 22 绘制了一个小型语言模型所采用的许多种调度及其相应测试性能。我们的结论是，只要 learning rate 总和足够大，并且调度包含预热阶段以及最终衰减至接近于零的 learning rate，那么 learning-rate schedule 的选择基本无关紧要。不同调度之间的变化似乎是统计噪声，可以粗略衡量不同训练运行之间的变化幅度。在较大模型上的实验表明，对于不同模型规模，不同随机种子之间最终测试损失的变化幅度大致恒定。

我们发现，较大的模型需要较小的 learning rate 来防止发散，而较小的模型则可以承受较大的 learning rate。为实现这一点，大多数训练运行采用了以下经验法则：

$$\text{LR}(N) \approx 0.003239 + -0.0001395 \log(N) \quad (\text{D.1})$$

我们预计这个公式还可以改进。它可能依赖于网络宽度，而这很可能由初始化尺度决定。当参数量 $N > 10^{10}$ 时，该公式也会失效。尽管如此，我们发现它对所考虑的模型已足够有效。

D.7 拟合细节与幂律质量

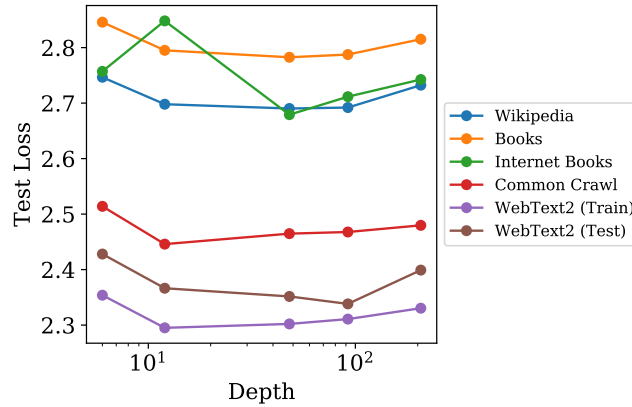


图 24 我们展示了约有 15 亿个参数的模型在一系列数据集上的评估结果。我们没有观察到深度对泛化的影响；泛化性能主要取决于训练分布上的性能。12 层模型在互联网图书数据集上发生了过拟合，因此我们展示了 early-stopped 性能；我们在其他实验中没有看到过这一令人意外的结果。

我们尝试了多种函数形式来拟合 $L(N)$, $L(C)$ 和 $L(D)$ ；与对数等其他函数相比，幂律拟合在定性上准确得多（见图 23）。

对于 $L(C)$ ，我们没有在拟合中纳入仅有 1 层的小型模型，因为从 1 层过渡到 2 层会使数据出现明显的凸起。对于 $L(N)$ ，我们同样没有在拟合中纳入仅有 1 层的极小模型，并且排除了尚未完全训练至收敛的最大模型。即使将它们纳入，拟合参数的变化也很小，并且无论是否纳入，该趋势在两个方向上都能很好地外推。

D.8 泛化与架构

在图 24 中，我们表明，当总参数量固定时，对其他数据分布的泛化不依赖于网络深度。它似乎只取决于训练分布上的性能。

图目录

1	简单幂律概述。	1
2	样本效率和计算效率示意图。	2
3	如何扩大模型规模、batch size 和串行步骤数	2
4	同时改变模型和数据大小，或模型和训练步骤数时的性能	3
5	性能对超参数调优的弱依赖性	5
6	包括或排除 embeddings 时的性能趋势比较	6
7	LSTM 与 Transformer 的性能比较	6
8	对其他测试数据集的泛化	7
9	过拟合的普适性	7
10	Critical batch size	9
11	性能与计算预算或参数更新次数的关系	10
12	在非最优模型上训练	11
13	实验计算趋势与校正后计算趋势的比较	11
14	最优模型规模和串行步数与计算预算的关系	12
15	计算趋势与数据趋势之间的矛盾	13
16	Early Stopping 下界及过拟合模型的训练曲线	18
17	通用 Transformer	18
18	Batch Size 扫描	18
19	从另一个角度看样本效率	19
20	性能对上下文中位置的幂律依赖	19
21	不同上下文位置的性能与模型规模之间的关系	19
22	learning-rate schedule 扫描	20
23	幂律拟合与对数拟合的比较	20
24	泛化与深度的关系	21

表目录

1	Transformer 的参数量和计算量	4
2	对 $L(N, D)$ 的拟合	8
3	对 $L(N, S)$ 的拟合	10
4	关键趋势方程	15
5	趋势拟合的关键参数	15
6	计算高效训练的趋势	15

参考文献

- [ACDE12] Eduardo G Altmann, Giampaolo Cristadoro, and Mirko Degli Esposti. On the origin of long-range correlations in texts. *Proceedings of the National Academy of Sciences*, 109(29):11582–11587, 2012.
- [AS17] Madhu S. Advani and Andrew M. Saxe. High-dimensional dynamics of generalization error in neural networks. *arXiv*, 2017, 1710.03667.
- [BB01] Michele Banko and Eric Brill. Scaling to very very large corpora for natural language disambiguation. In *Proceedings of the 39th annual meeting on association for computational linguistics*, pages 26–33. Association for Computational Linguistics, 2001.
- [BHMM18] Mikhail Belkin, Daniel Hsu, Siyuan Ma, and Soumik Mandal. Reconciling modern machine learning and the bias-variance trade-off. *arXiv*, 2018, 1812.11118.
- [Bia12] GÅŠrard Biau. Analysis of a random forests model. *Journal of Machine Learning Research*, 13(Apr):1063–1095, 2012.
- [CGRS19] Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers. *CoRR*, abs/1904.10509, 2019, 1904.10509. URL <http://arxiv.org/abs/1904.10509>.
- [DCLT18] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding, 2018, arXiv:1810.04805.
- [DGV⁺18] Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Lukasz Kaiser. Universal transformers. *CoRR*, abs/1807.03819, 2018, 1807.03819. URL <http://arxiv.org/abs/1807.03819>.
- [EP94] Werner Ebeling and Thorsten Pöschel. Entropy and long-range correlations in literary english. *EPL (Europhysics Letters)*, 26(4):241, 1994.
- [Fou] The Common Crawl Foundation. Common crawl. URL <http://commoncrawl.org>.
- [GARD18] Guy Gur-Ari, Daniel A. Roberts, and Ethan Dyer. Gradient descent happens in a tiny subspace. 2018, arXiv:1812.04754.
- [GJS⁺19] Mario Geiger, Arthur Jacot, Stefano Spigler, Franck Gabriel, Levent Sagun, Stéphane d’Ascoli, Giulio Biroli, Clément Hongler, and Matthieu Wyart. Scaling description of generalization with number of parameters in deep learning. *arXiv*, 2019, 1901.01608.
- [GKX19] Behrooz Ghorbani, Shankar Krishnan, and Ying Xiao. An investigation into neural net optimization via hessian eigenvalue density. *CoRR*, abs/1901.10159, 2019, 1901.10159. URL <http://arxiv.org/abs/1901.10159>.
- [Goo01] Joshua Goodman. A bit of progress in language modeling. *CoRR*, cs.CL/0108005, 2001. URL <http://arxiv.org/abs/cs.CL/0108005>.
- [GRK17] Scott Gray, Alec Radford, and Diederik P Kingma. Gpu kernels for block-sparse weights. *openai.com*, 2017.
- [HAD19] Joel Hestness, Newsha Ardalani, and Gregory Diamos. Beyond human-level accuracy: Computational challenges in deep learning. In *Proceedings of the 24th Symposium on Principles and Practice of Parallel Programming*, PPOPP ’19, pages 1–14, New York, NY, USA, 2019. ACM. doi:10.1145/3293883.3295710.
- [HCC⁺18] Yanping Huang, Yonglong Cheng, Dehao Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V. Le, and Zhifeng Chen. Gpipe: Efficient training of giant neural networks using pipeline parallelism. *CoRR*, abs/1811.06965, 2018, 1811.06965. URL <http://arxiv.org/abs/1811.06965>.
- [HNA⁺17] Joel Hestness, Sharan Narang, Newsha Ardalani, Gregory Diamos, Heewoo Jun, Hassan Kianinejad, Md. Mostofa Ali Patwary, Yang Yang, and Yanqi Zhou. Deep learning scaling is predictable, empirically, 2017, 1712.00409.
- [JGH18] Arthur Jacot, Franck Gabriel, and Clément Hongler. Neural tangent kernel: Convergence and generalization in neural networks. In *Advances in neural information processing systems*, pages 8571–8580, 2018.
- [KB14] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization, 2014, 1412.6980.
- [Kom19] Aran Komatsuzaki. One epoch is all you need, 2019, arXiv:1906.06669.

- [KSH12] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. Imagenet classification with deep convolutional neural networks. In *Proceedings of the 25th International Conference on Neural Information Processing Systems - Volume 1*, NIPS'12, pages 1097–1105, USA, 2012. Curran Associates Inc. URL <http://dl.acm.org/citation.cfm?id=2999134.2999257>.
- [LCG⁺19] Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, and Radu Soricut. Albert: A lite bert for self-supervised learning of language representations, 2019, 1909.11942.
- [LOG⁺19] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized BERT pretraining approach. *CoRR*, abs/1907.11692, 2019, 1907.11692. URL <http://arxiv.org/abs/1907.11692>.
- [LSP⁺18] Peter J. Liu, Mohammad Saleh, Etienne Pot, Ben Goodrich, Ryan Sepassi, Lukasz Kaiser, and Noam Shazeer. Generating wikipedia by summarizing long sequences. *arXiv:1801.10198 [cs]*, 2018, 1801.10198. URL <http://arxiv.org/abs/1801.10198>.
- [LT16] Henry W Lin and Max Tegmark. Criticality in formal languages and statistical physics. *arXiv preprint arXiv:1606.06737*, 2016.
- [LXS⁺19] Jaehoon Lee, Lechao Xiao, Samuel S. Schoenholz, Yasaman Bahri, Roman Novak, Jascha Sohl-Dickstein, and Jeffrey Pennington. Wide neural networks of any depth evolve as linear models under gradient descent, 2019, arXiv:1902.06720.
- [MKAT18] Sam McCandlish, Jared Kaplan, Dario Amodei, and OpenAI Dota Team. An empirical model of large-batch training, 2018, arXiv:1812.06162.
- [Pap18] Vardan Papyan. The full spectrum of deep net hessians at scale: Dynamics with sample size. *CoRR*, abs/1811.07062, 2018, 1811.07062. URL <http://arxiv.org/abs/1811.07062>.
- [RNSS18] Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. Improving language understanding by generative pre-training. URL https://s3-us-west-2.amazonaws.com/openai-assets/research-covers/language-unsupervised/language_understanding_paper.pdf, 2018.
- [RRBS19a] Jonathan S. Rosenfeld, Amir Rosenfeld, Yonatan Belinkov, and Nir Shavit. A constructive prediction of the generalization error across scales, 2019, 1909.12673.
- [RRBS19b] Jonathan S. Rosenfeld, Amir Rosenfeld, Yonatan Belinkov, and Nir Shavit. A constructive prediction of the generalization error across scales, 2019, arXiv:1909.12673.
- [RSR⁺19] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer, 2019, arXiv:1910.10683.
- [RWC⁺19] Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. *openai.com*, 2019.
- [SCP⁺18] Noam Shazeer, Youlong Cheng, Niki Parmar, Dustin Tran, Ashish Vaswani, Penporn Koanantakool, Peter Hawkins, HyounJoong Lee, Mingsheng Hong, Cliff Young, Ryan Sepassi, and Blake Hechtman. Mesh-tensorflow: Deep learning for supercomputers, 2018, 1811.02084.
- [SHB15] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. *CoRR*, 2015, 1508.07909.
- [SLA⁺18] Christopher J. Shallue, Jaehoon Lee, Joe Antognini, Jascha Sohl-Dickstein, Roy Frostig, and George E. Dahl. Measuring the effects of data parallelism on neural network training, 2018, arXiv:1811.03600.
- [SS18] Noam Shazeer and Mitchell Stern. Adafactor: Adaptive learning rates with sublinear memory cost. *CoRR*, abs/1804.04235, 2018, 1804.04235. URL <http://arxiv.org/abs/1804.04235>.
- [THK18] Stefan Thurner, Rudolf Hanel, and Peter Klimek. *Introduction to the theory of complex systems*. Oxford University Press, 2018.
- [TL19] Mingxing Tan and Quoc V. Le. Efficientnet: Rethinking model scaling for convolutional neural networks. *CoRR*, abs/1905.11946, 2019, 1905.11946. URL <http://arxiv.org/abs/1905.11946>.

- [VSP⁺17] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems 30*, pages 5998–6008. Curran Associates, Inc., 2017. URL <http://papers.nips.cc/paper/7181-attention-is-all-you-need.pdf>.
- [VWB16] Andreas Veit, Michael Wilber, and Serge Belongie. Residual networks behave like ensembles of relatively shallow networks, 2016, arXiv:1605.06431.
- [Was06] Larry Wasserman. *All of nonparametric statistics*. Springer Science & Business Media, 2006.
- [WPN⁺19] Alex Wang, Yada Pruksachatkun, Nikita Nangia, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. Superglue: A stickier benchmark for general-purpose language understanding systems, 2019, 1905.00537.
- [WRH17] Yu-Xiong Wang, Deva Ramanan, and Martial Hebert. Growing a brain: Fine-tuning by increasing model capacity. *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Jul 2017. doi:10.1109/cvpr.2017.323.
- [WYL19] Wei Wen, Feng Yan, and Hai Li. Autogrow: Automatic layer growing in deep convolutional networks, 2019, 1906.02909.
- [YDY⁺19] Zhilin Yang, Zihang Dai, Yiming Yang, Jaime Carbonell, Ruslan Salakhutdinov, and Quoc V. Le. Xlnet: Generalized autoregressive pretraining for language understanding, 2019, arXiv:1906.08237.
- [ZK16] Sergey Zagoruyko and Nikos Komodakis. Wide residual networks. *Proceedings of the British Machine Vision Conference 2016*, 2016. doi:10.5244/c.30.87.
- [ZKZ⁺15] Yukun Zhu, Ryan Kiros, Rich Zemel, Ruslan Salakhutdinov, Raquel Urtasun, Antonio Torralba, and Sanja Fidler. Aligning books and movies: Towards story-like visual explanations by watching movies and reading books. *2015 IEEE International Conference on Computer Vision (ICCV)*, Dec 2015. doi:10.1109/iccv.2015.11.
- [ZLN⁺19] Guodong Zhang, Lala Li, Zachary Nado, James Martens, Sushant Sachdeva, George E. Dahl, Christopher J. Shallue, and Roger B. Grosse. Which algorithmic choices matter at which batch sizes? insights from a noisy quadratic model. *CoRR*, abs/1907.04164, 2019, 1907.04164. URL <http://arxiv.org/abs/1907.04164>.